

Compiler Design - Interview Overview

A complete, interview-focused guide covering how compilers and interpreters work, their phases, and the key runtime translation strategies used in real systems.

Table of Contents

1. [Compiler vs Interpreter](#)
2. [Compiler Phases](#)
3. [Front End, Middle End and Back End](#)
4. [Source Language and Target Language](#)
5. [Symbol Table](#)
6. [Error Handling](#)
7. [One-Pass vs Multi-Pass Compiler](#)
8. [Just-In-Time \(JIT\) Compilation](#)
9. [Ahead-Of-Time \(AOT\) Compilation](#)
10. [Bootstrapping a Compiler](#)
11. [Self-Hosting Compilers](#)

1. Compiler vs Interpreter

1. Overview

Definition. A **compiler** is a program that translates an entire source program written in one language (the source language) into an equivalent program in another language (usually machine code or bytecode) *before* the program runs. An **interpreter** is a program that reads the source program and executes it directly, statement by statement, *without* producing a separate standalone executable.

Put simply: a compiler is like translating a whole book from English to Hindi and handing you the finished translated book. An interpreter is like a live human translator standing next to you, translating each sentence out loud as it is spoken.

Why it matters.

- It determines *when* translation happens (before running vs during running).
- It affects **execution speed**, **startup time**, **error reporting**, **portability**, and **memory footprint**.
- It is the foundational mental model for understanding JIT, AOT, bytecode VMs, and modern language runtimes.

Where it is used in real systems.

- **Compiled:** C, C++, Rust, Go, Swift produce native machine code.
- **Interpreted:** classic Python (CPython), Ruby (MRI), JavaScript (early engines), Bash, PHP.
- **Hybrid (both):** Java and C# compile to bytecode, then interpret/JIT it. Modern Python compiles to bytecode too, then interprets it.

Why interviewers ask about it. It is the single most common compiler-design opener. It quickly reveals whether you understand the difference between *translation time* and *execution time*, and whether you know that most real languages today are hybrids, not purely one or the other.

2. Core Idea

The core distinction is **when translation happens and what artifact is produced**.

- Compiler: **source code** → (translate once, ahead of time) → **machine code / bytecode** → run the produced artifact many times.
- Interpreter: **source code** → (read + translate + execute each construct, every run) → results directly.

Intuition. A compiler front-loads all the work. It spends time upfront analyzing and optimizing so that execution is fast later. An interpreter does the work lazily, translating as it goes, so it starts fast but each run repeats the translation cost.

Real-world analogy.

- Compiler = a **recipe translation service**. You send an English recipe, they mail you back the full French recipe. Cooking (running) is then fast and repeatable.
- Interpreter = a **live cooking-show translator** who reads each English step and immediately tells the French chef what to do. Nothing is saved; next time you cook, you need the translator again.

Small example.

```
// C - compiled
#include <stdio.h>
int main() { printf("Hello\n"); return 0; }
```

```
$ gcc hello.c -o hello    # translation happens now (compile time)
$ ./hello                # execution - no compiler involved
Hello
```

```
# Python - interpreted
print("Hello")
```

```
$ python hello.py        # translation + execution happen together every run
Hello
```

Step-by-step (compiler): read source → check syntax and types → optimize → emit machine code file → OS loads and runs that file. **Step-by-step (interpreter):** read one statement → analyze it → execute it → move to next statement → repeat.

3. Important Subtopics

3.1 Bytecode and Virtual Machines (the hybrid model)

- **What:** Source is compiled to a compact intermediate form called **bytecode**, which a **virtual machine (VM)** then interprets or JIT-compiles.
- **Why it matters:** Combines portability (one bytecode runs everywhere the VM runs) with better speed than pure source interpretation.
- **Example:** Java `.java` → `javac` → `.class` bytecode → JVM runs it. Python `.py` → `.pyc` bytecode → CPython VM runs it.
- **Interview angle:** "Is Java compiled or interpreted?" Correct answer: *both* - compiled to bytecode, then interpreted and JIT-compiled by the JVM.

3.2 Error detection timing

- **What:** Compilers catch many errors (type errors, undeclared variables) at compile time before any code runs. Interpreters catch errors only when execution reaches the faulty line.
- **Why it matters:** Compile-time errors are found earlier and more cheaply; interpreter errors can hide in rarely executed branches.
- **Example:** A type mismatch in an `if` branch never taken will be flagged by a compiler but may never surface in an interpreter.
- **Interview angle:** "Where do you find a bug in an unexecuted branch?" Compilers can; pure interpreters cannot.

3.3 Execution speed vs startup speed

- **What:** Compiled native code runs fastest but pays compile time upfront. Interpreters start instantly but run slower per statement.
- **Why it matters:** Drives language choice - scripting/glue favors interpreters, performance-critical systems favor compilers.
- **Interview angle:** "Why is C faster than Python?" Native compilation + no per-statement translation overhead + static types.

3.4 Portability

- **What:** Compiled native binaries are tied to a CPU/OS. Interpreted source and bytecode are portable as long as the interpreter/VM exists on the target.
- **Why it matters:** "Write once, run anywhere" (Java) comes from bytecode + VM, not from the source language itself.

4. Real-World Example

Web browser JavaScript engines (V8 in Chrome/Node.js). JavaScript began as a purely interpreted language. Today V8 is a hybrid: it parses JS, generates bytecode, interprets it with the **Ignition** interpreter for fast startup, and *simultaneously* watches which functions run often ("hot" code). Hot functions are handed to the **TurboFan** optimizing compiler, which produces fast native machine code at runtime (this is JIT). This is exactly why a webpage loads and reacts instantly (interpreter startup) yet number-crunching loops still run fast (JIT to native). The compiler-vs-interpreter tradeoff is resolved by using *both*, adaptively.

5. Diagrams / Mental Models

COMPILER (ahead of time)

source.c → [Compiler] → machine code → [CPU runs] → output
(once) (many times, fast)

INTERPRETER (during execution)

source.py → [Interpreter: read → translate → execute, per statement] → output
(every run repeats this work)

HYBRID (bytecode + VM)

source.java → [javac] → bytecode.class → [JVM: interpret + JIT] → output

Dimension	Compiler	Interpreter
When translated	Before execution	During execution
Output artifact	Executable / bytecode	None (runs directly)
Execution speed	Fast	Slower
Startup time	Slower (must compile)	Fast
Error reporting	All at compile time	At the failing line, at runtime
Portability of artifact	Low (native)	High (source/bytecode)
Memory at runtime	Lower	Higher (interpreter resident)
Examples	C, C++, Rust, Go	CPython, Ruby MRI, Bash

6. Common Interview Questions

Q1. What is the difference between a compiler and an interpreter? Answer: A compiler translates the whole program to another language (usually machine code) before execution and produces a standalone artifact; an interpreter translates and executes the program statement by statement at runtime without producing a separate executable. Key points expected: translation timing, artifact produced, speed vs startup tradeoff. Common mistake: saying "compiler is fast, interpreter is slow" without explaining *why* (per-statement re-translation, no upfront optimization).

Q2. Is Java compiled or interpreted? Answer: Both. `javac` compiles source to platform-independent bytecode; the JVM then interprets that bytecode and JIT-compiles hot paths to native code. Key points: bytecode, JVM, JIT. Common mistake: answering only "compiled" or only "interpreted".

Q3. Which reports errors earlier, and why? Answer: A compiler, because it analyzes the entire program (syntax, types, scope) before running, so it can flag errors in code that may never execute. Common mistake: forgetting the "unexecuted branch" insight.

Q4. Why does compiled code generally run faster? Answer: Translation and optimization happen once, ahead of time; there is no per-statement translation overhead at runtime, and static type information enables

aggressive optimization. Common mistake: attributing speed only to "machine code" and ignoring the removal of repeated translation.

Q5. Can the same language be both compiled and interpreted? Answer: Yes. C normally is compiled but interpreters exist (Cling). Python is normally interpreted but can be compiled (Cython, Nuitka). Compiled-vs-interpreted is a property of the *implementation*, not the language. Common mistake: treating it as an intrinsic language property.

Q6. What is bytecode and why use it? Answer: A compact, platform-independent intermediate instruction set produced by a compiler and executed by a VM. It gives portability plus faster execution than re-parsing source each time. Common mistake: confusing bytecode with machine code.

Q7. What are the advantages of an interpreter? Answer: Fast startup, easier debugging with an interactive REPL, platform portability of source, and dynamic features (eval, runtime code generation). Common mistake: listing only disadvantages.

Q8. What are disadvantages of a compiler? Answer: Slower edit-compile-run cycle, produces platform-specific binaries, less runtime flexibility, larger build tooling. Common mistake: claiming compilers have "no downsides".

Q9. What is a REPL and which model does it fit? Answer: Read-Eval-Print Loop; an interactive interpreter that reads an expression, evaluates it, prints the result, and loops. It fits the interpreter model and is why interpreted languages are great for exploration. Common mistake: thinking a REPL cannot use compilation (modern REPLs may JIT).

Q10. Give one system that uses both a compiler and interpreter together. Answer: The JVM or V8: source is compiled to bytecode, the bytecode is interpreted for fast startup, and hot code is JIT-compiled to native for speed. Common mistake: not naming a concrete adaptive system.

7. Deep-Dive Questions

D1. Why do modern interpreters compile to bytecode instead of interpreting the AST directly? Bytecode is a flat, linear instruction stream that is far cheaper to dispatch than walking a tree of AST node objects for every operation. Bytecode dispatch loops are cache-friendly and enable techniques like threaded dispatch. It is also a stable, compact serialization (Python caches `.pyc`), avoiding re-parsing on every run.

D2. What is the "warm-up" problem in hybrid runtimes? JIT-based runtimes start by interpreting, then must observe code long enough to decide what to optimize and then compile it. During this warm-up window, performance is lower and CPU/memory are spent profiling and compiling. Short-lived programs may never reach peak performance, which is why some workloads prefer AOT.

D3. How does an interpreter handle a syntax error deep in the file? Most real interpreters still *parse the entire file first* (a compile-to-bytecode step), so syntax errors are reported before any execution. Purely line-by-line interpreters would only fail when reaching the bad line. This is why Python reports a `SyntaxError` before running any statement.

D4. Can compilation and interpretation give different results for the same program? Ideally no, but they can diverge on undefined/unspecified behavior, floating-point rounding under aggressive optimization, evaluation-order assumptions, or timing-dependent code. A correct implementation must preserve the language's defined semantics regardless of strategy.

D5. Why might you deliberately choose an interpreter for a performance-sensitive product? Fast startup, tiny memory footprint, portability, and simpler deployment can outweigh raw throughput - for example short-lived CLI tools, serverless functions with cold-start constraints, or embedded scripting where a JIT's memory cost is unacceptable.

8. Comparison Tables

Compiler vs Interpreter vs Hybrid (JIT)

Feature	Pure Compiler (AOT)	Pure Interpreter	Hybrid / JIT
Translation time	Before run	During run	Before run (to bytecode) + during run (hot paths)
Peak speed	High	Low	High (after warm-up)
Startup	Slow build	Instant	Fast (interpret first)
Portability of artifact	Low	High	High (bytecode)
Adapts to runtime data	No	No	Yes (profile-guided)
Examples	C, Rust	Bash, early JS	JVM, V8, .NET

Translation artifact comparison

Artifact	Produced by	Runs on
Native machine code	Compiler	CPU directly
Bytecode	Bytecode compiler	Virtual machine
Source (no artifact)	none	Interpreter

9. Common Mistakes

- Believing a language is *inherently* compiled or interpreted (it is the implementation that decides).
- Saying "Java is compiled" and stopping there (it is bytecode + JVM + JIT).
- Confusing bytecode with native machine code.
- Assuming interpreters never catch errors before running (most parse fully first).
- Thinking JIT is a kind of interpreter (it is a *compiler* that runs at runtime).
- Claiming compiled code is *always* faster in practice (JITs can beat AOT using runtime profiling; short programs may not amortize compile cost).

10. Edge Cases / Special Cases

- **Transpilers** (source-to-source compilers, e.g. TypeScript → JavaScript, Babel) are compilers whose *target* is another high-level language, not machine code.
- **Ahead-of-time compilation of "interpreted" languages** exists (Cython, PyPy's tracing JIT, GraalVM native image for Java).

- **eval / dynamic code**: interpreted/JIT runtimes can execute code generated at runtime; a pure AOT binary generally cannot without embedding a compiler.
- **Cold start vs steady state**: benchmarks can mislead because a JIT looks slow at start but fast at steady state.
- **Self-modifying / reflective code** is easy in interpreters, hard in static compilers.

11. How to Explain in Interview

"A compiler translates the whole program ahead of time into machine code or bytecode and produces an artifact you run later, so execution is fast but the build step is upfront. An interpreter reads and executes the program directly, statement by statement, so it starts instantly but runs slower because translation is repeated. In practice most modern languages are hybrids: Java and Python compile to bytecode and then a virtual machine interprets it, and hot code paths are JIT-compiled to native code for speed. So it is less 'which one' and more 'when does translation happen and what artifact do we keep'."

12. Quick Revision Notes

- **Compiler**: translate whole program before running → artifact (machine code/bytecode).
- **Interpreter**: translate + execute per statement at runtime → no artifact.
- **Hybrid**: source → bytecode → VM interprets + JITs hot code (JVM, V8, CPython+, .NET).
- Compiled = faster run, slower build, less portable binary. Interpreted = fast start, slower run, portable source.
- Compiled/interpreted is a property of the **implementation**, not the language.
- **Trap**: "Is Java compiled or interpreted?" → *both*.
- **Trap**: JIT is a compiler that runs at runtime, not an interpreter.
- Bytecode != machine code.

13. Practice Tasks

1. Compile `hello.c` with `gcc -o hello hello.c`, then run `./hello`. Note that the compiler is not involved in the second step.
2. Run the same logic in Python and time both first-run and repeated runs; observe startup vs steady-state differences.
3. Disassemble Python bytecode: `python -c "import dis; dis.dis(lambda x: x+1)"` and read the bytecode instructions.
4. Compile a Java file with `javac`, inspect the `.class`, then run with `java`. Explain each step's role.
5. Introduce a type error inside an `if` branch that never runs, in both C and Python. Observe which implementation catches it.
6. Write a two-line tree-walking interpreter for `+` and `*` over a small AST in Python to feel per-node dispatch.

14. Final Cheat Sheet

- **Core definition**: Compiler = translate all before run (artifact produced). Interpreter = translate + run per statement (no artifact).
- **Why it matters**: Governs speed, startup, portability, and when errors are found.
- **Most asked**: Difference? Is Java compiled or interpreted (both)? Why is compiled faster? What is bytecode?

- **Common comparisons:** Compiler vs Interpreter vs JIT; native code vs bytecode vs source.
 - **One-line answer:** "A compiler translates the whole program ahead of time into a runnable artifact; an interpreter translates and executes it statement by statement at runtime - and most modern languages do both via bytecode and a VM."
-

2. Compiler Phases

1. Overview

Definition. The **phases of a compiler** are the ordered stages through which source code is transformed into target code. The classic pipeline is: **Lexical Analysis** → **Syntax Analysis** → **Semantic Analysis** → **Intermediate Code Generation** → **Code Optimization** → **Code Generation**, supported throughout by the **Symbol Table** and **Error Handler**.

Why it matters.

- It is the backbone structure of every compiler and interpreter.
- Each phase has a clear input, output, and responsibility, which makes the huge problem of "translate a language" tractable and modular.
- Understanding phases explains where specific errors come from (a missing semicolon vs an undeclared variable vs a type mismatch).

Where it is used in real systems. GCC, Clang/LLVM, `javac`, the Roslyn C# compiler, TypeScript's `tsc`, and even SQL query engines all follow this phase structure (parsing → semantic checks → intermediate representation → optimization → code/plan generation).

Why interviewers ask about it. It tests whether you can decompose a complex system into stages, name each stage's job, and map real error messages to the phase that produces them. "Which phase catches an undeclared variable?" is a classic discriminating question.

2. Core Idea

A compiler is a **pipeline of transformations**. Each phase takes the output of the previous phase, does one well-defined job, and passes a richer/lower-level representation forward.

Intuition. You cannot understand a sentence all at once. First you split it into words (lexing), then check grammar (parsing), then check meaning (semantics), then translate to a neutral idea (intermediate code), polish the wording (optimization), and finally speak it in the target language (code generation).

Real-world analogy - translating a novel:

1. **Lexical:** break text into words and punctuation (tokens).
2. **Syntax:** verify the words form grammatical sentences (parse tree).
3. **Semantic:** verify sentences make sense (a "green idea" that "sleeps furiously" is grammatical but meaningless - type/scope check).
4. **Intermediate code:** rewrite each sentence as a simple language-neutral meaning.
5. **Optimization:** remove redundancy, tighten phrasing.
6. **Code generation:** write it out in the final target language.

Small example. For `position = initial + rate * 60`:

- **Tokens:** `id(position) = id(initial) + id(rate) * num(60)`
- **Parse tree:** an assignment whose right side is `initial + (rate * 60)` (precedence handled).
- **Semantic:** all ids declared; `60` may be converted to float (type coercion via `inttfloat`).
- **Intermediate (three-address code):**

```
t1 = inttfloat(60)
t2 = rate * t1
t3 = initial + t2
position = t3
```

- **Optimized:**

```
t1 = rate * 60.0      ; fold int-to-float at compile time
position = initial + t1
```

- **Target (assembly-like):**

```
LDF  R2, rate
MULF R2, R2, #60.0
LDF  R1, initial
ADDF R1, R1, R2
STF  position, R1
```

3. Important Subtopics

3.1 Lexical Analysis (Scanner)

- **What:** Reads the character stream and groups it into **tokens** (keywords, identifiers, literals, operators, punctuation). Strips whitespace and comments.
- **Why it matters:** Turns raw text into meaningful units so the parser deals with tokens, not characters.
- **Example:** `int x = 10;` → `KEYWORD(int) ID(x) OP(=) NUM(10) SEMI`.
- **Interview angle:** "Which phase catches an illegal character like `@` in C?" - the lexer. Tool: **Lex/Flex**. Uses regular expressions and finite automata.

3.2 Syntax Analysis (Parser)

- **What:** Checks that the token sequence follows the language grammar (usually a context-free grammar) and builds a **parse tree / abstract syntax tree (AST)**.
- **Why it matters:** Enforces structure and operator precedence; the AST drives all later phases.
- **Example:** Missing semicolon or unbalanced braces → syntax error here.
- **Interview angle:** Top-down (LL, recursive descent) vs bottom-up (LR, LALR) parsing. Tool: **Yacc/Bison**.

3.3 Semantic Analysis

- **What:** Checks meaning using the AST and symbol table: type checking, declaration-before-use, scope resolution, function argument counts, type coercions.
- **Why it matters:** Grammatically valid code can still be meaningless (`"abc" * 3.5` in a typed language).
- **Example:** Assigning a string to an int variable, or calling an undeclared function.
- **Interview angle:** "Which phase catches type mismatches and undeclared variables?" - semantic analysis. It heavily uses the symbol table.

3.4 Intermediate Code Generation

- **What:** Produces a machine-independent intermediate representation (IR) such as **three-address code (TAC)**, quadruples, or LLVM IR.
- **Why it matters:** Decouples the front end from the back end so one front end can target many machines and optimizations are written once.
- **Example:** `a = b + c * d` → `t1 = c * d; a = b + t1`.
- **Interview angle:** Why an IR at all? Portability + reusable optimizations (the "N languages × M machines" problem).

3.5 Code Optimization

- **What:** Transforms the IR to run faster or smaller while preserving meaning: constant folding, dead code elimination, common subexpression elimination, loop-invariant code motion, strength reduction.
- **Why it matters:** Directly affects runtime performance and code size.
- **Example:** `x = 3 * 4` → `x = 12` (constant folding); removing a variable that is never used (dead code elimination).
- **Interview angle:** Machine-independent (on IR) vs machine-dependent (register allocation) optimization; it is optional but valuable.

3.6 Code Generation

- **What:** Translates optimized IR into target code (assembly/machine code), handling register allocation and instruction selection.
- **Why it matters:** Final step producing runnable output; quality of register allocation strongly affects speed.
- **Example:** IR `t3 = initial + t2` → `ADD R1, R2, R3`.
- **Interview angle:** Register allocation (graph coloring), instruction selection, this is the machine-dependent back end.

3.7 Symbol Table and Error Handler (cross-cutting)

- **What:** Not a sequential phase but used by *all* phases. The **symbol table** stores names, types, scopes; the **error handler** reports and recovers from errors in each phase.
- **Why it matters:** Enables scope/type checking and graceful multi-error reporting.

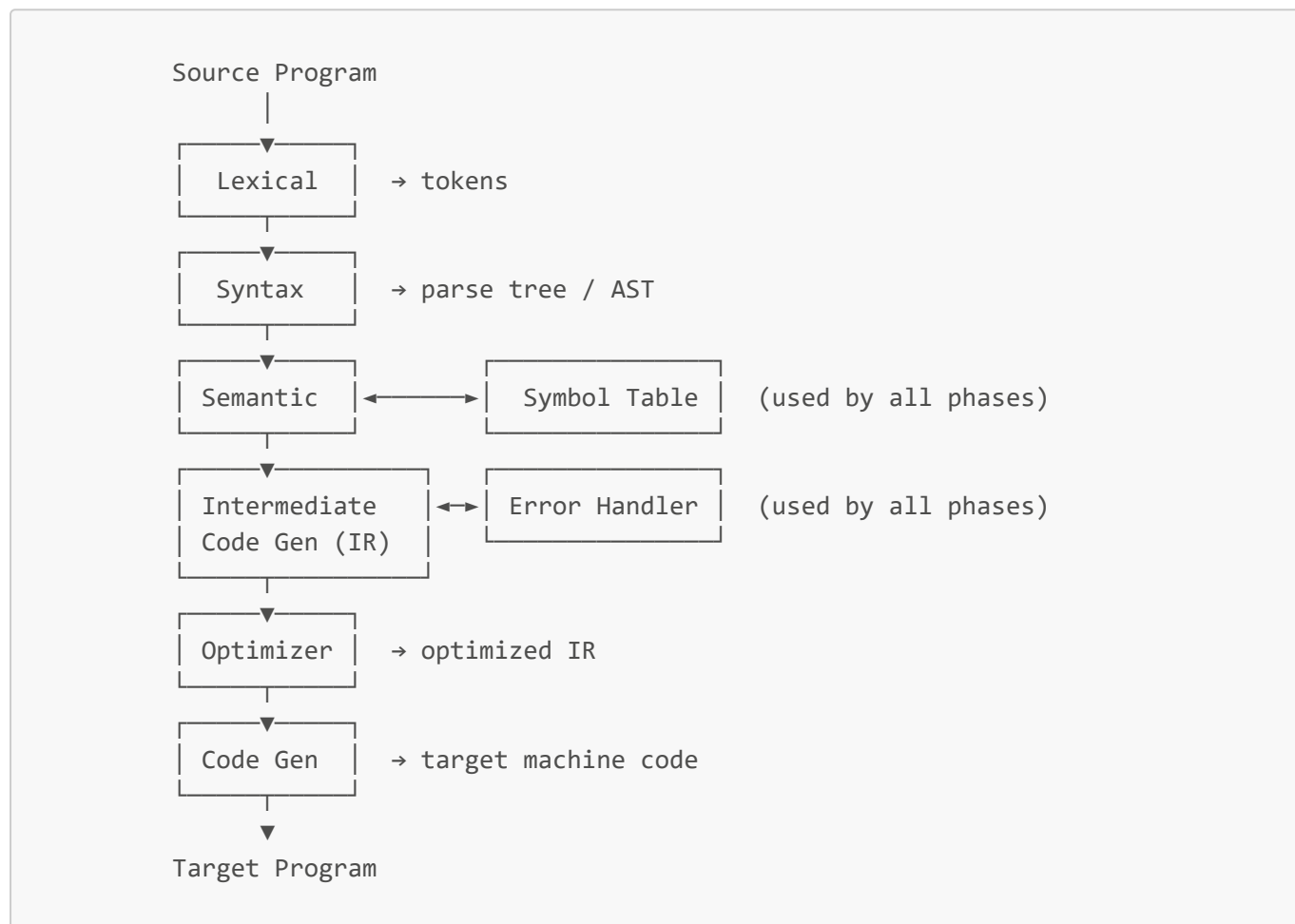
4. Real-World Example

A database SQL engine mirrors compiler phases. When you run `SELECT name FROM users WHERE age > 30`:

1. **Lexing:** split into tokens `SELECT`, `name`, `FROM`, `users`, ...

2. **Parsing:** build a query parse tree; reject `SELECT` or missing `FROM`.
3. **Semantic analysis (binding):** confirm table `users` and column `name/age` exist and `age` is comparable to `30` (uses the catalog = symbol table).
4. **Intermediate representation:** a logical query plan (relational algebra tree).
5. **Optimization:** the query optimizer reorders joins, pushes down the `WHERE age > 30` filter, chooses indexes.
6. **Code/plan generation:** produce a physical execution plan the engine runs. This is why understanding compiler phases directly transfers to understanding query planners.

5. Diagrams / Mental Models



Phase	Input	Output	Catches
Lexical	characters	tokens	illegal characters, bad tokens
Syntax	tokens	AST/parse tree	grammar errors (missing <code>;</code> , unbalanced <code>{}</code>)
Semantic	AST + symbol table	annotated AST	type errors, undeclared/undefined names
Intermediate	annotated AST	IR (e.g. TAC)	-
Optimization	IR	optimized IR	-
Code Gen	optimized IR	machine code	-

6. Common Interview Questions

- Q1. What are the phases of a compiler?** Answer: Lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, code generation - plus the symbol table and error handler used across all phases. Common mistake: omitting the symbol table/error handler or scrambling the order.
- Q2. Which phase detects an undeclared variable?** Answer: Semantic analysis, using the symbol table (the name is a valid token and syntactically fine, but has no declaration). Common mistake: saying lexical or syntax.
- Q3. Which phase detects a missing semicolon?** Answer: Syntax analysis (parser), because the token stream violates the grammar. Common mistake: saying lexical analysis.
- Q4. Which phase detects an illegal symbol like # (in a language that disallows it)?** Answer: Lexical analysis, it cannot form a valid token.
- Q5. What is the difference between a parse tree and an AST?** Answer: A parse tree records every grammar rule including punctuation and intermediate non-terminals; an AST is a condensed tree keeping only semantically meaningful nodes (operators/operands), dropping redundant tokens. Common mistake: treating them as identical.
- Q6. What is three-address code?** Answer: An intermediate representation where each instruction has at most one operator and up to three addresses (two operands, one result), e.g. `t1 = a + b`. It simplifies optimization and translation.
- Q7. Why generate an intermediate representation at all?** Answer: To decouple front end from back end, enabling `N` languages and `M` targets with `N+M` components instead of `N*M`, and to write optimizations once on the IR.
- Q8. Name three code optimizations.** Answer: Constant folding, dead code elimination, common subexpression elimination (also loop-invariant motion, strength reduction). Common mistake: confusing optimization with code generation.
- Q9. Is code optimization mandatory?** Answer: No. A compiler is correct without it (`-O0`); optimization improves speed/size but is optional and must preserve semantics.
- Q10. What are the two components used across all phases?** Answer: The symbol table and the error handler.
- Q11. Which phases form the "analysis" (front end) vs "synthesis" (back end)?** Answer: Analysis = lexical, syntax, semantic (understands and breaks down the source). Synthesis = intermediate code, optimization, code generation (builds the target).

7. Deep-Dive Questions

- D1. How do lexer and parser interact - batch or on demand?** Typically the parser drives the lexer on demand: the parser calls `getNextToken()` when it needs the next token, so lexing and parsing interleave rather than the lexer producing the whole token list first. This saves memory and supports streaming.
- D2. Where does operator precedence get resolved?** In syntax analysis, through the grammar (precedence/associativity rules or precedence-climbing/Pratt parsing), which is why `a + b * c` parses as `a + (b * c)` and the AST already encodes the correct order for later phases.

D3. Can semantic errors ever be caught at parse time? Some can be folded into the grammar, but classic semantic errors (type mismatch, undeclared use, wrong argument count) need context (the symbol table) that a context-free grammar cannot express, so they belong to semantic analysis.

D4. What is the difference between machine-independent and machine-dependent optimization?

Machine-independent optimizations act on the IR (constant folding, dead code, CSE) and apply to any target. Machine-dependent optimizations (register allocation, instruction scheduling, peephole) exploit the specific target CPU during/after code generation.

D5. Why is register allocation hard, and how is it modeled? There are far fewer physical registers than temporaries, so the compiler must decide which values live in registers vs memory. It is commonly modeled as **graph coloring** on an interference graph (variables that are live simultaneously cannot share a register); it is NP-hard, so heuristics are used.

8. Comparison Tables

Analysis (Front End) vs Synthesis (Back End)

Aspect	Analysis phases	Synthesis phases
Phases	lexical, syntax, semantic	intermediate, optimization, code gen
Job	understand & validate source	construct target code
Machine dependence	independent	partly dependent (code gen)
Output	annotated AST + symbol table	machine code

Parse Tree vs AST

Feature	Parse Tree	AST
Detail	every grammar symbol, punctuation	only meaningful nodes
Size	larger	smaller
Use	shows derivation	used by later phases

Error phase mapping

Error	Phase
@ illegal char	Lexical
missing ;, }	Syntax
undeclared variable, type mismatch	Semantic
divide-by-zero at runtime	none (runtime, not a compile phase)

9. Common Mistakes

- Mixing up which phase catches which error (semicolon = syntax, not lexical; undeclared var = semantic, not syntax).

- Forgetting the symbol table and error handler are cross-cutting, not sequential phases.
- Thinking optimization is mandatory or part of code generation.
- Confusing parse tree with AST.
- Believing intermediate code is optional fluff (it is central to portability and reusable optimization).
- Reversing the analysis/synthesis grouping.

10. Edge Cases / Special Cases

- **Single combined front-end pass:** small compilers merge lexing, parsing, and semantic checks; the phases are still conceptually distinct.
- **Multiple IR levels:** real compilers (LLVM) use several IRs (high-level AST → LLVM IR → machine IR).
- **Runtime errors are not a compiler phase:** null dereference or divide-by-zero occur during execution, not during any compile phase.
- **Preprocessing:** in C, macro expansion happens before lexical analysis (a separate preprocessor pass).
- **Error recovery** lets later phases keep running after an error to report multiple issues at once.

11. How to Explain in Interview

"A compiler is a pipeline. Lexical analysis turns characters into tokens. Syntax analysis checks grammar and builds an AST. Semantic analysis uses the symbol table to check types, scopes, and declarations. Then intermediate code generation produces a machine-independent representation like three-address code, the optimizer improves it, and code generation emits target machine code. Throughout, a symbol table and an error handler support every phase. The first three phases are the analysis/front end that understands the source; the last three are the synthesis/back end that builds the target. A neat way to remember it: a missing semicolon is syntax, an undeclared variable is semantic, an illegal character is lexical."

12. Quick Revision Notes

- Order: **Lexical** → **Syntax** → **Semantic** → **Intermediate** → **Optimization** → **Code Gen**.
- Cross-cutting: **Symbol Table** + **Error Handler**.
- Front end (analysis) = lexical + syntax + semantic. Back end (synthesis) = intermediate + optimization + code gen.
- Lexer → tokens (Flex, regex/FA). Parser → AST (Bison, CFG). Semantic → type/scope checks.
- IR example = three-address code (`t1 = a + b`). Why IR: decouple front/back end, reuse optimizations.
- **Traps:** semicolon = syntax; undeclared var = semantic; `@` = lexical; runtime divide-by-zero = no phase.
- Optimization is optional; must preserve meaning.

13. Practice Tasks

1. Take `d = a + b * c - 5;` and by hand produce: tokens, AST (with precedence), three-address code, and one optimization.
2. Classify these errors by phase: (a) `int 3x = 5;` (b) `if (x > 0 { (c) y = undeclaredVar + 1; (d) int a = "hi";`.
3. Write regular expressions for identifiers, integer literals, and floating-point literals.
4. Use `gcc -S file.c` to view generated assembly, and `-O2` vs `-O0` to see optimization differences.
5. Draw both the parse tree and the AST for `a = b + c` and mark what the AST drops.
6. Trace `x = 2 * 3 + 4` through constant folding and show the optimized IR.

14. Final Cheat Sheet

- **Core definition:** Ordered stages transforming source to target: lexical, syntax, semantic, intermediate, optimization, code gen, with symbol table + error handler across all.
 - **Why it matters:** Modular design; explains where each error is caught; foundation of all compilers and query engines.
 - **Most asked:** List the phases; which phase catches undeclared var / missing semicolon / illegal char; parse tree vs AST; what is TAC.
 - **Common comparisons:** Analysis vs synthesis; parse tree vs AST; machine-independent vs dependent optimization.
 - **One-line answer:** "A compiler is a pipeline - lexing, parsing, semantic analysis (front end), then intermediate code, optimization, and code generation (back end) - with a symbol table and error handler supporting every stage."
-

3. Front End, Middle End and Back End

1. Overview

Definition. Modern compilers are organized into three logical parts:

- **Front end:** language-specific. Reads source, checks it, and produces an intermediate representation (IR). (lexical + syntax + semantic analysis)
- **Middle end:** language- and machine-independent. Optimizes the IR.
- **Back end:** target-machine-specific. Turns optimized IR into machine code (instruction selection, register allocation, scheduling).

Why it matters. This split solves the "**N languages by M machines**" **explosion**. Instead of writing a separate compiler for every (language, CPU) pair (**N*M** compilers), you write **N** front ends and **M** back ends that all meet at a common IR (**N+M** components).

Where it is used in real systems. **LLVM** is the textbook example: Clang (C/C++), Rust, and Swift front ends all emit **LLVM IR**; the LLVM middle end optimizes it; back ends emit x86, ARM, RISC-V, WebAssembly. **GCC** uses GIMPLE/RTL similarly. .NET separates C#/F# front ends from the CIL/JIT back end.

Why interviewers ask about it. It shows you understand *modular compiler architecture* and the practical reason IR exists. A great follow-up test: "You have 5 languages and 4 CPUs - how many compiler components with vs without a shared IR?"

2. Core Idea

Put a **narrow, well-defined waist (the IR)** in the middle. Everything above the waist is about *understanding the language*; everything below is about *targeting the machine*. The middle end works only on the IR and knows about neither.

Intuition. It is a translation hub. Many source languages funnel *into* one neutral representation; from that neutral representation, code fans *out* to many machines. The neutral representation is the interchange format.

Real-world analogy - a currency exchange. Instead of exchange rates between every pair of currencies ($N*N$), convert every currency to a hub currency (say USD = the IR) and back out. You maintain $2N$ rates, not N^2 . The IR is the hub currency of compilation.

Small example (LLVM-style).



Add a new language: write one front end, get all CPUs free. Add a new CPU: write one back end, get all languages free.

Step by step: front end parses + type-checks + lowers to IR → middle end runs target-independent optimizations (inlining, constant propagation, dead code) → back end selects instructions, allocates registers, schedules, and emits assembly for the chosen target.

3. Important Subtopics

3.1 The Front End

- **What:** Lexical, syntax, and semantic analysis; produces AST then lowers to IR. Owns everything language-specific.
- **Why it matters:** Isolates language quirks so the rest of the compiler is language-agnostic.
- **Example:** Clang parses C++ templates and emits LLVM IR; the back end never sees a template.
- **Interview angle:** "What is language-specific in a compiler?" - the front end.

3.2 The Middle End (Optimizer)

- **What:** Runs target- and language-independent optimizations on the IR: constant folding/propagation, dead code elimination, common subexpression elimination, function inlining, loop optimizations.
- **Why it matters:** Optimizations are written *once* and benefit every language and every target.
- **Example:** LLVM's **opt** passes; GCC's tree-SSA passes.
- **Interview angle:** Why put optimization in the middle? So it is shared across all front ends and back ends.

3.3 The Back End (Code Generator)

- **What:** Instruction selection, register allocation (graph coloring), instruction scheduling, and machine-specific (peephole) optimization; emits assembly/machine code. Owns everything machine-specific.
- **Why it matters:** Isolates CPU details so front ends and the optimizer stay portable.
- **Example:** LLVM x86 vs ARM back ends from the same optimized IR.
- **Interview angle:** "What is machine-specific?" - the back end (registers, instruction set, calling convention).

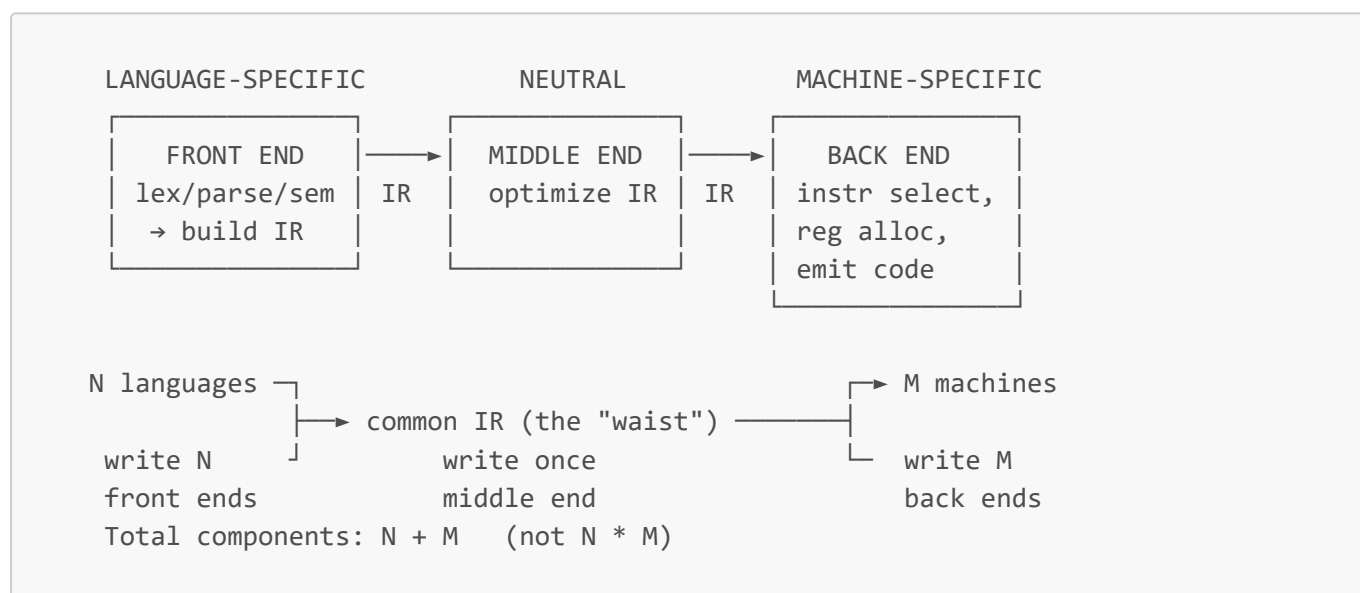
3.4 The Intermediate Representation (the waist)

- **What:** The shared contract between the three parts, e.g. LLVM IR, GIMPLE, Java bytecode, .NET CIL, three-address code.
- **Why it matters:** Its stability and generality are what make the **N+M** decomposition possible.
- **Interview angle:** A good IR is language-neutral, machine-neutral, and easy to optimize.

4. Real-World Example

Rust and Swift did not each build a full compiler. Both reused LLVM. The Rust compiler (**rustc**) front end does borrow-checking and lowers to its own MIR and then to LLVM IR; Swift's front end handles Swift semantics and emits SIL then LLVM IR. From there, LLVM's shared middle end optimizes and its back ends emit x86, ARM (for iPhones), WebAssembly, and more. This is why a brand-new language can target dozens of CPUs on day one: it only needs a front end that reaches LLVM IR. Browsers use the same idea in reverse - Emscripten compiles C/C++ through LLVM to WebAssembly so native code runs in a browser.

5. Diagrams / Mental Models



Part	Depends on	Job	Owns
Front end	source language	analyze, build IR	language rules, types, syntax
Middle end	neither	optimize IR	target-independent optimizations
Back end	target machine	emit machine code	registers, ISA, calling convention

6. Common Interview Questions

Q1. What are the front end, middle end, and back end of a compiler? Answer: Front end is language-specific (lex/parse/semantic → IR); middle end optimizes the IR and is language/machine-independent; back end is target-specific (instruction selection, register allocation → machine code). Common mistake: putting optimization in the front end or back end only.

Q2. Why split a compiler this way? Answer: To reuse components: **N** front ends + **M** back ends meeting at one IR give **N+M** pieces instead of **N*M**, and optimizations are written once. Common mistake: not quantifying

the $N+M$ vs $N*M$ benefit.

Q3. Which part is language-specific and which is machine-specific? Answer: Front end = language-specific; back end = machine-specific; middle end = neither.

Q4. What connects the three parts? Answer: The intermediate representation (IR), e.g. LLVM IR, bytecode, three-address code.

Q5. Where does register allocation happen? Answer: In the back end, because it depends on the target CPU's register set.

Q6. Where does function inlining or constant propagation happen? Answer: In the middle end (target-independent optimization on the IR).

Q7. Give a real example of this architecture. Answer: LLVM - Clang/Rust/Swift front ends → LLVM IR → shared optimizer → x86/ARM/WASM back ends.

Q8. You have 5 languages and 4 CPU targets. How many components with and without a shared IR? Answer: Without IR: $5*4 = 20$ full compilers. With IR: 5 front ends + 4 back ends = 9 components.

Q9. Can two languages share a back end? Answer: Yes - if both lower to the same IR (as C++ and Rust do to LLVM IR), they share the entire middle and back end.

Q10. Is the middle end mandatory? Answer: No; a front end can hand IR straight to a back end. The middle end is where optional optimization lives.

7. Deep-Dive Questions

D1. What makes a good IR? It should be language-neutral (not leak C or Java specifics), machine-neutral (not assume x86), simple and regular (easy to analyze), and expressive enough to preserve semantics needed for optimization. LLVM IR uses SSA form and typed values for exactly this.

D2. Why is SSA (Static Single Assignment) used in the middle end? In SSA each variable is assigned exactly once, so def-use chains are explicit. This makes optimizations like constant propagation, dead code elimination, and value numbering simpler and faster. Phi-nodes reconcile values at control-flow merges.

D3. Can optimization ever be machine-dependent, contradicting the "middle end is neutral" idea? Yes - some optimizations (instruction scheduling, register allocation, target-specific peephole) must live in the back end because they depend on the CPU. The middle end holds only the target-independent ones; back ends add machine-specific passes.

D4. How does this architecture enable cross-compilation? Because the back end is a swappable component parameterized by target, one compiler on an x86 machine can select an ARM back end and emit ARM code. The front end and middle end are unchanged; only the back end differs.

D5. Where do language-specific optimizations go if the middle end is neutral? They go in the front end (or a language-specific IR before the common IR). For example Rust's borrow analysis and some devirtualization happen in `rustc`'s MIR before lowering to LLVM IR, since LLVM does not understand Rust semantics.

8. Comparison Tables

Front vs Middle vs Back end

Feature	Front end	Middle end	Back end
Depends on language?	Yes	No	No
Depends on machine?	No	No	Yes
Main job	analyze + build IR	optimize IR	generate machine code
Example passes	lexing, parsing, type check	inlining, DCE, CSE	instruction selection, register allocation
Reused across	one language	all languages + targets	one target family

With vs Without shared IR (N languages, M targets)

	Components needed	Add a language	Add a target
No shared IR	$N * M$	build M compilers	build N compilers
Shared IR	$N + M$	build 1 front end	build 1 back end

9. Common Mistakes

- Thinking optimization belongs to the front end or the back end alone (its portable part is the middle end).
- Believing the split is about *phases* rather than *reusability/portability*.
- Assuming the back end handles types and scopes (that is the front end).
- Forgetting that some optimizations (register allocation, scheduling) are inherently back-end.
- Not being able to state the $N+M$ vs $N*M$ argument, which is the whole point.
- Confusing IR with source code or with final machine code.

10. Edge Cases / Special Cases

- Multiple IRs:** real compilers use several (Clang AST → LLVM IR → MachineIR; GCC GENERIC → GIMPLE → RTL).
- Retargetable back ends:** one compiler can emit many CPUs by swapping back ends (cross-compilation).
- Front-end-only tools:** linters and type checkers (like `tsc --noEmit`) use just the front end.
- JIT reuse:** a JIT can reuse the same middle/back end at runtime (LLVM used as a JIT).
- Fat/"universal" binaries:** run the back end multiple times for different targets and bundle the outputs.
- WebAssembly as a target:** the back end can emit WASM, making the browser "just another machine".

11. How to Explain in Interview

"Modern compilers are split into three parts connected by an intermediate representation. The front end is language-specific: it lexes, parses, and type-checks the source and lowers it to IR. The middle end is neutral: it

runs target-independent optimizations on the IR, written once and shared by everyone. The back end is machine-specific: it does instruction selection, register allocation, and emits code for a particular CPU. The big win is reuse - with N languages and M targets you need $N+M$ components instead of $N*M$, which is exactly why Rust and Swift could target dozens of CPUs by reusing LLVM's middle and back ends."

12. Quick Revision Notes

- Front end = language-specific (analysis $\rightarrow$ IR). Middle end = neutral (optimize IR). Back end = machine-specific (IR $\rightarrow$ machine code).
- The IR is the "waist" connecting all three (LLVM IR, bytecode, TAC).
- **Key win:** $N + M$ components vs $N * M$ compilers.
- Register allocation + instruction selection = back end. Inlining, constant propagation, DCE, CSE = middle end.
- LLVM is the canonical example (Clang/Rust/Swift $\rightarrow$ LLVM IR $\rightarrow$ x86/ARM/WASM).
- **Trap:** optimization is not solely front/back end; the portable optimizer is the middle end.
- Swappable back end = cross-compilation.

13. Practice Tasks

1. Compute components for 5 languages and 4 targets, with and without a shared IR; explain the saving.
2. Run `clang -S -emit-llvm file.c -o file.ll` and read the LLVM IR; identify what is language-neutral.
3. Run `opt -O2 file.ll -S` and diff before/after to see middle-end optimizations.
4. Run `llc -march=arm file.ll` vs default target to see the back end swap (cross-compilation).
5. List which of these are front/middle/back end: type checking, register allocation, inlining, parsing, instruction scheduling.
6. Explain how Emscripten uses this architecture to run C++ in a browser via WASM.

14. Final Cheat Sheet

- **Core definition:** Front end (language-specific analysis $\rightarrow$ IR), middle end (neutral IR optimization), back end (machine-specific code generation), joined by a shared IR.
- **Why it matters:** $N+M$ reusable components instead of $N*M$ full compilers; portability and cross-compilation.
- **Most asked:** What are the three parts; which is language vs machine specific; why split; the $N+M$ argument; example (LLVM).
- **Common comparisons:** Front vs middle vs back end; with vs without shared IR.
- **One-line answer:** "Split the compiler at a neutral IR: language-specific front ends and machine-specific back ends meet in a shared optimizer, so N languages and M targets cost $N+M$ components, not $N*M$."

4. Source Language and Target Language

1. Overview

Definition. In any translation, the **source language** is the language the input program is written in (what the compiler reads), and the **target language** is the language the compiler produces (what it writes). A compiler is

fundamentally a function: `compiler : Source → Target`.

Why it matters.

- These two labels define *what a compiler does*: translate from one to the other while preserving meaning.
- The target is not always machine code - it can be bytecode, assembly, or even another high-level language (transpilers).
- The pairing determines the tooling: `C → x86 assembly`, `Java → JVM bytecode`, `TypeScript → JavaScript`.

Where it is used in real systems.

- `gcc`: source = C, target = machine code (via assembly).
- `javac`: source = Java, target = JVM bytecode.
- `tsc` (TypeScript): source = TypeScript, target = JavaScript (a transpiler).
- `nvcc`: source = CUDA C++, target = PTX/GPU code.

Why interviewers ask about it. It checks that you know the target need not be machine code, that you understand transpilers/cross-compilers, and that you can articulate the invariant a translator must preserve: **semantic equivalence** (the target program behaves like the source program).

2. Core Idea

A compiler maps a program written in the source language to an equivalent program in the target language. "Equivalent" is the crucial word: for every valid input, the target program must produce the same observable behavior the source program's semantics require.

Intuition. Source is "what the human wrote"; target is "what the machine (or next tool) runs". The compiler is the bridge, and the bridge must not change the meaning.

Real-world analogy. Translating a legal contract from English (source) to French (target). The words change completely, but every clause must carry the *same legal meaning*. A translation that reads beautifully but changes an obligation is a bug.

Small example.

- Source (C): `int sq(int x){ return x*x; }`
- Target (x86-64 assembly, conceptually):

```
sq:
    mov eax, edi    ; x
    imul eax, eax   ; x * x
    ret
```

The source and target look nothing alike, yet `sq(5)` must yield `25` in both. Same meaning, different language.

Levels of "target".

High-level source	→ another high-level language	(transpiler: TS → JS)
High-level source	→ bytecode	(javac: Java → JVM bytecode)
High-level source	→ assembly	(gcc -S: C → asm)
Assembly	→ machine code	(assembler: asm → binary)

3. Important Subtopics

3.1 Target = Machine Code

- **What:** The target is native binary instructions for a specific CPU/OS.
- **Why it matters:** Runs directly on hardware, fastest execution, but not portable.
- **Example:** `gcc hello.c -o hello` produces an ELF/PE binary.
- **Interview angle:** Native target ⇒ platform-specific ⇒ needs cross-compilation for other CPUs.

3.2 Target = Bytecode

- **What:** The target is a portable virtual-machine instruction set.
- **Why it matters:** "Compile once, run anywhere" - portability plus a VM that can JIT.
- **Example:** `javac` → JVM bytecode; C# → CIL.
- **Interview angle:** Bytecode is still a target language, just for a virtual machine instead of physical hardware.

3.3 Target = Another High-Level Language (Transpiler / Source-to-Source)

- **What:** Both source and target are high-level; a.k.a. transcompiler.
- **Why it matters:** Lets you use a nicer/newer language while running on an existing ecosystem.
- **Example:** TypeScript → JavaScript, Babel (modern JS → older JS), CoffeeScript → JS, C++ → C historically (Cfront).
- **Interview angle:** "Is a transpiler a compiler?" Yes - it just has a high-level target.

3.4 Cross-Compilation (host vs target machine)

- **What:** Compiling on one machine (the host) to run on a different machine (the target).
- **Why it matters:** You build phone/embedded binaries on a laptop; the source language is unchanged, only the target machine differs.
- **Example:** Building ARM firmware on an x86 PC.
- **Interview angle:** Distinguish *source/target language* from *host/target machine*, they are different axes.

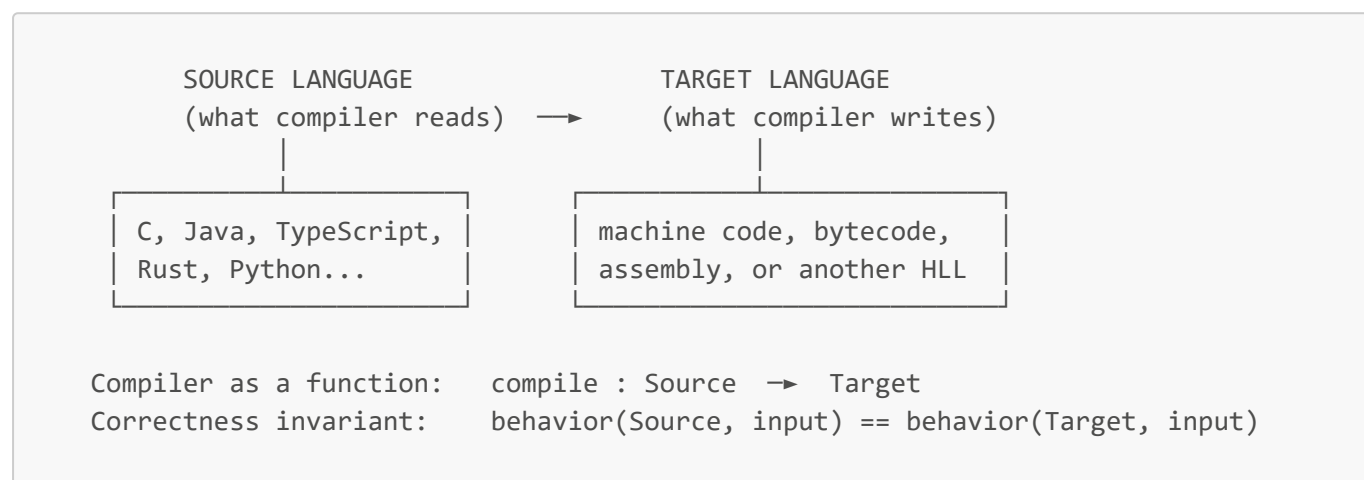
3.5 Semantic Equivalence (the invariant)

- **What:** The target program must preserve the observable behavior mandated by the source language's semantics.
- **Why it matters:** This is the correctness contract of a compiler; optimizations are legal only if they preserve it.
- **Example:** Reordering `a + b + c` is legal for integers only if it does not change results (careful with floats/overflow).

4. Real-World Example

TypeScript in a modern web app. Developers write in TypeScript (source language) for static types and better tooling, but browsers cannot run TypeScript. The `tsc` compiler *transpiles* it to JavaScript (target language), stripping types and lowering newer syntax (like `async/await` or optional chaining) into equivalent older JavaScript when needed. The emitted JS is then delivered to browsers, where V8 treats *that* JS as its own source and compiles it to bytecode/native. So one artifact is simultaneously a compiler's target (of `tsc`) and another compiler's source (of V8) - a clear demonstration that "source" and "target" are roles, not fixed properties.

5. Diagrams / Mental Models



Tool	Source language	Target language
gcc / clang	C / C++	machine code (native)
gcc -S	C	assembly
javac	Java	JVM bytecode
tsc	TypeScript	JavaScript
Babel	modern JS	older JS
nvcc	CUDA C++	PTX / GPU code
assembler	assembly	machine code

6. Common Interview Questions

Q1. What are the source and target languages of a compiler? Answer: The source language is the language of the input program the compiler reads; the target language is the language the compiler produces (machine code, bytecode, assembly, or another high-level language). Common mistake: assuming the target is always machine code.

Q2. Is the target language always machine code? Answer: No. It can be bytecode (`javac` → JVM), assembly (`gcc -S`), or another high-level language (TypeScript → JavaScript).

Q3. What is a transpiler? Answer: A source-to-source compiler whose target is another high-level language, e.g. TypeScript → JavaScript or Babel converting modern JS to older JS. Common mistake: saying a transpiler is "not a real compiler".

- Q4. What is the essential correctness property a compiler must maintain?** Answer: Semantic equivalence - the target program must produce the behavior the source program's semantics require, for all inputs.
- Q5. What is cross-compilation?** Answer: Compiling on a host machine to produce a target-machine binary that runs on a different architecture, e.g. building ARM code on an x86 laptop.
- Q6. Difference between source/target language and host/target machine?** Answer: Source/target language is *what is translated*; host/target machine is *where the compiler runs vs where the output runs*. They are independent axes.
- Q7. Give three different target languages a C program could be compiled to.** Answer: x86 machine code, ARM machine code (cross-compile), and LLVM IR or WebAssembly.
- Q8. Why compile TypeScript to JavaScript instead of machine code?** Answer: To run inside existing JavaScript engines/browsers, reusing that ecosystem; the JS engine handles the final step to native.
- Q9. Can the same program be both a target and a source?** Answer: Yes - `tsc` outputs JS (its target), which is then the source for V8.
- Q10. Does changing the target language change the front end?** Answer: Ideally no - the front end depends on the source language; the back end depends on the target. Changing target changes the back end.
- ## 7. Deep-Dive Questions
- D1. What exactly does "semantic equivalence" allow a compiler to change?** It may change everything unobservable: instruction choice, register use, evaluation order (when it does not affect results), even remove code, as long as the program's *defined* observable behavior (outputs, effects, defined ordering) is preserved. Undefined behavior in the source frees the compiler to assume it never happens.
- D2. Why can floating-point reassociation break equivalence but integer reassociation often not?** Floating-point addition is not associative due to rounding, so $(a+b)+c$ may differ from $a+(b+c)$. Integers are associative (ignoring overflow rules), so reordering is safe. Compilers therefore restrict FP reordering unless `-ffast-math` is set, trading strict equivalence for speed.
- D3. How does the choice of target language affect portability vs performance?** A native-machine-code target maximizes performance but is tied to one CPU/OS. A bytecode target maximizes portability (any VM host) at some runtime cost. A high-level target (transpiler) maximizes ecosystem reuse but defers real code generation to another compiler.
- D4. Is an assembler a compiler? Is a disassembler?** An assembler translates assembly (source) to machine code (target), so it is a (very thin) compiler by the source→target definition, though usually classified separately as it is near one-to-one. A disassembler goes machine code → assembly; it is a *decompiler*-style tool, translating in the reverse direction.
- D5. What is a decompiler in this framing?** A translator whose source is a low-level language (machine code/bytecode) and target is a higher-level language (C/Java-like). It inverts the usual direction and is lossy because names, types, and structure were discarded during the original compilation.

8. Comparison Tables

Kinds of target language

Target type	Example tool	Runs on	Portability	Speed
Native machine code	gcc, rustc	CPU directly	Low	Highest
Bytecode	javac, csc	Virtual machine	High	High (with JIT)
Assembly	gcc -S	assembler then CPU	Low	Highest
High-level (transpile)	tsc, Babel	another compiler/runtime	Depends on host lang	Depends

Language axis vs machine axis

	Source language	Target language
Meaning	input to compiler	output of compiler
Example	Java	JVM bytecode
	Host machine	Target machine
Meaning	where compiler runs	where output runs
Example	x86 laptop	ARM phone

9. Common Mistakes

- Assuming the target language is always native machine code.
- Thinking a transpiler is not a real compiler.
- Confusing source/target *language* with host/target *machine*.
- Believing a compiler may change program behavior "a little" for speed (it must preserve defined semantics).
- Forgetting that bytecode and assembly are also target languages.
- Not realizing one tool's target can be another tool's source.

10. Edge Cases / Special Cases

- **Multi-target compilers:** the same source can be compiled to several targets (WASM, x86, ARM) by swapping back ends.
- **Undefined behavior:** the source's UB lets the compiler produce "surprising" but technically legal target code.
- **Self-targeting:** a compiler can target its own source language (identity/pretty-printer) or another dialect.
- **Fat binaries:** one file containing machine code for multiple targets (Apple universal binaries).
- **Decompilation:** reversing target back to a high-level language is lossy (lost names/comments/types).
- **Assemblers/linkers** sit at the very bottom of the source→target chain.

11. How to Explain in Interview

"The source language is what the compiler reads and the target language is what it writes. People assume the target is always machine code, but it can be bytecode like JVM bytecode, assembly, or even another high-level language - that last case is a transpiler, like TypeScript to JavaScript. Whatever the target, the one rule a compiler must never break is semantic equivalence: for every input, the target program must behave the way the source program's semantics require. And note the language axis is different from the machine axis - cross-compilation means building on one machine, the host, to run on another, the target, without changing the source language."

12. Quick Revision Notes

- **Source language** = compiler input; **target language** = compiler output.
- Target can be: native machine code, assembly, bytecode, or another high-level language (transpiler).
- **Compiler = function:** **Source** → **Target**, invariant = **semantic equivalence**.
- Transpiler = source-to-source compiler (TS → JS, Babel).
- **Two independent axes:** source/target *language* vs host/target *machine* (cross-compilation).
- **Trap:** "target = machine code always" is wrong.
- One tool's target (TS→JS) can be another tool's source (JS→V8).

13. Practice Tasks

1. Run `gcc -S hello.c` and read the assembly (the target) alongside the C (the source).
2. Compile the same C to two targets: native and WebAssembly (via Emscripten); compare.
3. Transpile a small TypeScript file with `tsc` and diff the generated JavaScript against the source.
4. Cross-compile a "hello world" for ARM from an x86 machine (`arm-linux-gnueabi-hf-gcc`); note language stays C.
5. Write a tiny "compiler" in Python that translates simple arithmetic expressions (source) to reverse-Polish/stack code (target).
6. Take `(a + b) + c` in floats and integers; argue whether a compiler may reassociate each while preserving equivalence.

14. Final Cheat Sheet

- **Core definition:** Source language = what the compiler reads; target language = what it produces (machine code, assembly, bytecode, or another HLL).
- **Why it matters:** Defines what a compiler is and the semantic-equivalence contract it must preserve.
- **Most asked:** Is target always machine code (no); what is a transpiler; source/target language vs host/target machine; cross-compilation.
- **Common comparisons:** Native vs bytecode vs high-level target; language axis vs machine axis.
- **One-line answer:** "A compiler translates a source-language program into a semantically equivalent target-language program, where the target may be machine code, bytecode, assembly, or even another high-level language."

5. Symbol Table

1. Overview

Definition. A **symbol table** is a data structure maintained by the compiler that stores information about every identifier (name) in the source program - variables, functions, classes, types, parameters, labels - along with attributes such as **type, scope, memory location/offset, storage class, and parameter list**. It is the compiler's "database of names".

Why it matters.

- It is the shared memory that lets phases communicate: the parser inserts declarations, the semantic analyzer looks them up for type/scope checks, and the code generator uses them for addresses/offsets.
- Almost every meaningful compile-time check (undeclared variable, type mismatch, redeclaration, wrong argument count) is a symbol-table lookup.

Where it is used in real systems. Every compiler (GCC, Clang, `javac`), every interpreter, IDEs (for autocomplete, go-to-definition, rename), linkers (symbol resolution across object files), and databases (the **system catalog** is essentially a symbol table for tables/columns).

Why interviewers ask about it. It connects data structures (hash tables, scoping) with compiler semantics. Great questions include "what data structure would you use and why", "how do you implement nested scopes", and "which errors are caught via the symbol table".

2. Core Idea

The symbol table answers one question fast and correctly: **"Given this name in this scope, what do I know about it?"** It must support efficient **insert** (on declaration) and **lookup** (on use), and it must respect **scope** so the same name can mean different things in different regions.

Intuition. It is a phone book for identifiers. When you see a name being used, you look it up to find its "contact info" (type, where it lives in memory, whether it is even declared).

Real-world analogy - a company directory with departments. "Alex" in the Engineering department is a different person than "Alex" in Sales. To resolve "Alex", you check the current department first, then broader ones. Departments = scopes; the directory = symbol table; the lookup order = scope resolution (inner to outer).

Small example.

```
int x;           // global
void f(int a) { // a is a parameter
    int x;       // local x shadows global x
    x = a + 1;   // which x? the inner one
}
```

Symbol table (conceptually, per scope):

Scope	Name	Type	Kind	Notes
global	x	int	variable	
global	f	int(int)	function	

Scope	Name	Type	Kind	Notes
f (local)	a	int	parameter	
f (local)	x	int	variable	shadows global x

Step by step for `x = a + 1;`: look up `x` → found in local scope (inner wins) → look up `a` → parameter → check both are `int` → generate code writing to local `x`'s address.

3. Important Subtopics

3.1 What is stored (attributes)

- **What:** name, type, scope/level, memory offset or address, storage class (static/auto/extern), size, dimensions (arrays), parameter list (functions), value (constants).
- **Why it matters:** Later phases need each of these (semantic analysis needs type; code gen needs offset).
- **Interview angle:** "What fields does a symbol table entry hold?" - be ready to list type, scope, offset, kind.

3.2 Operations

- **What:** `insert(name, attributes)` on declaration, `lookup(name)` on use, plus scope `enterScope()/exitScope()`.
- **Why it matters:** Correctness and speed of the whole compiler hinge on these being right and fast.
- **Interview angle:** Insert must detect **redeclaration** within the same scope.

3.3 Data structures used

- **What:** hash table (most common, average $O(1)$), balanced BST (ordered, $O(\log n)$), linked list (simple, $O(n)$), or a stack of hash tables for scopes.
- **Why it matters:** Choice trades speed vs simplicity vs ordering.
- **Example:** GCC/Clang use hash-based maps.
- **Interview angle:** "Best data structure and why?" - hash table for average $O(1)$ insert/lookup.

3.4 Scope management (nested scopes)

- **What:** Handling blocks, functions, and shadowing. Common approaches: a **stack of scope tables** (push on entering a block, pop on leaving) or a single hash table with scope tags.
- **Why it matters:** Correctly resolves which declaration a name refers to and enforces visibility.
- **Example:** Inner `x` shadows outer `x`; leaving the block discards the inner entry.
- **Interview angle:** Static (lexical) scoping resolves names by program structure; lookup goes inner scope outward.

3.5 Scope resolution and shadowing

- **What:** Lookup searches the innermost scope first, then enclosing scopes, up to global.
- **Why it matters:** Defines variable visibility and shadowing semantics.
- **Interview angle:** Difference between shadowing (legal, inner hides outer) and redeclaration (illegal, same scope twice).

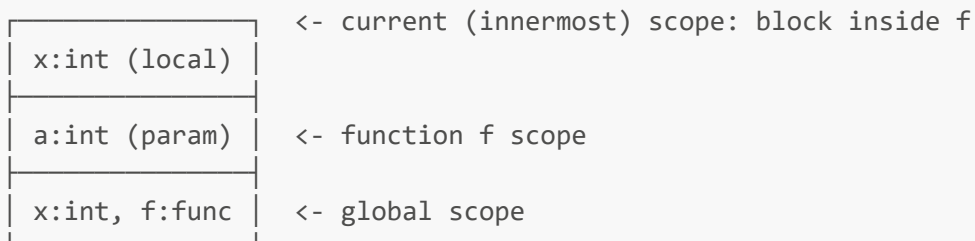
4. Real-World Example

IDE "Go to Definition" and "Rename Symbol". When you Ctrl-click a variable in VS Code, the language server has built a symbol table (and scope tree) for your file/project. Clicking resolves the name in its scope to the exact declaration entry, which stores the definition location - that is how it jumps you there. "Rename" is safe precisely because the symbol table distinguishes the two different `xs` in different scopes, so renaming the inner one does not touch the outer one. Autocomplete lists names visible in the current scope by walking the scope chain outward. The same structure powers a **database's system catalog**, which stores table and column metadata so the query binder can check that `SELECT age FROM users` refers to real columns.

5. Diagrams / Mental Models

Scope stack (a stack of symbol tables):

push on entering block, pop on leaving



Lookup("x") searches TOP-DOWN (inner → outer), returns first match.

Hash-table entry layout:

name →hash→ bucket → [name | type | scope | offset | kind | ...]

Operation	When	Complexity (hash table)
insert	on declaration	O(1) average
lookup	on use	O(1) average
enterScope / exitScope	block boundaries	O(1)

6. Common Interview Questions

Q1. What is a symbol table and why is it needed? Answer: A compiler data structure storing identifiers and their attributes (type, scope, offset, kind); it is needed so phases can share name information for type checking, scope resolution, and code generation. Common mistake: describing only "variable names" and ignoring attributes and scope.

Q2. What information does a symbol table entry contain? Answer: Name, type, scope/level, memory offset or address, storage class, size, and for functions the parameter list and return type.

Q3. Which data structure is best for a symbol table and why? Answer: A hash table, for average $O(1)$ insert and lookup; BSTs give ordering at $O(\log n)$; lists are simplest but $O(n)$. Common mistake: saying "array" without considering lookup cost.

Q4. Which phases use the symbol table? Answer: All major phases - it is populated during lexical/syntax analysis, heavily used in semantic analysis (type/scope checks), and read in code generation (offsets/addresses).

Q5. How are nested scopes handled? Answer: With a stack of scope tables (push on block entry, pop on exit) or a single table with scope tags; lookup proceeds from innermost to outermost scope.

Q6. What is shadowing vs redeclaration? Answer: Shadowing is an inner-scope declaration hiding an outer one (legal). Redeclaration is declaring the same name twice in the *same* scope (an error the symbol table detects on insert).

Q7. How does the symbol table help detect an undeclared variable? Answer: On use, lookup fails in all enclosing scopes, so the semantic analyzer reports "undeclared identifier".

Q8. How does it help detect type errors? Answer: Lookup returns each operand's type; the semantic analyzer compares them against the operator's expected types.

Q9. What happens to a scope's entries when the scope ends? Answer: They are removed/hidden (the scope table is popped), so those names are no longer visible.

Q10. Is the symbol table used at runtime? Answer: Usually not for compiled languages (names become addresses/offsets), but debug builds and interpreters keep symbol info; dynamic languages retain it at runtime.

Q11. How do you handle function overloading in a symbol table? Answer: Key entries by name *plus* signature (parameter types), so multiple functions with the same name but different parameters coexist.

7. Deep-Dive Questions

D1. How would you implement scoping efficiently without popping entries on every block exit? Use a single hash table where each name maps to a *stack* of entries; on **enterScope** you record a marker, on declaration you push onto the name's stack, and on **exitScope** you pop everything declared since the marker. Lookup reads the top of each name's stack. This gives $O(1)$ lookup and cheap scope exit.

D2. Static (lexical) vs dynamic scoping - how does each affect symbol resolution? Static scoping resolves names by the program's textual structure (enclosing blocks), decidable at compile time using the scope tree - this is what most languages use. Dynamic scoping resolves names by the runtime call stack (the most recent binding), which cannot be fully resolved by a static symbol table and needs runtime lookup.

D3. How are records/structs and classes represented? Each aggregate type gets its own nested symbol table (or scope) mapping member names to types and *offsets within the object*. A field access **obj.field** looks up **field** in the struct's member table to get its offset, which the code generator adds to the base address.

D4. Why does the code generator still need the symbol table? To turn names into concrete storage: local variables become stack offsets, globals become labels/addresses, struct fields become offsets, and function calls need signatures and calling conventions - all stored as symbol attributes.

D5. How do linkers use symbol tables across compilation units? Each object file has a symbol table listing *defined* and *undefined* (external) symbols. The linker matches undefined references in one object to definitions in another, resolving addresses. Unresolved symbols cause "undefined reference" link errors; duplicate definitions cause "multiple definition" errors.

8. Comparison Tables

Data structures for a symbol table

Structure	Insert	Lookup	Ordered?	Notes
Hash table	$O(1)$ avg	$O(1)$ avg	No	most common choice
Balanced BST	$O(\log n)$	$O(\log n)$	Yes	good if ordered traversal needed
Linked list	$O(1)$	$O(n)$	No	simplest, small tables only
Stack of hash tables	$O(1)$	$O(1)$ per level	No	natural for nested scopes

Shadowing vs Redeclaration

	Shadowing	Redeclaration
Scopes	different (inner hides outer)	same scope
Legal?	Yes	No (error)
Detected by	scope resolution	insert collision check

Static vs Dynamic scoping

Feature	Static (lexical)	Dynamic
Resolved by	program text/structure	runtime call stack
Compile-time decidable	Yes	No
Common in	most languages (C, Java)	older Lisp, some shells

9. Common Mistakes

- Thinking the symbol table only stores variable names (it stores types, scopes, offsets, functions, etc.).
- Believing it is used in one phase only (it spans lexing to code generation, and linking).
- Confusing shadowing (legal) with redeclaration (error).
- Using an unordered array and ignoring lookup cost.
- Forgetting scope management, so nested/same-name variables resolve incorrectly.
- Assuming names still exist at runtime in compiled code (usually replaced by addresses).

10. Edge Cases / Special Cases

- **Forward references:** using a function before its definition needs either two passes or forward declarations so the name is in the table when used.

- **Recursion:** a function must be inserted into the symbol table *before* processing its body so it can call itself.
- **Overloading/generics:** entries keyed by name + signature; templates/generics may need specialized entries.
- **Blocks with same-name variables** in sibling scopes are independent entries.
- **Labels and goto** are their own namespace, separate from variables.
- **Namespaces/modules** add another layer of qualified lookup (`std::vector`).

11. How to Explain in Interview

"A symbol table is the compiler's database of identifiers. For each name - variable, function, type - it stores attributes like type, scope, memory offset, and kind. Every phase touches it: the parser inserts declarations, the semantic analyzer looks names up to check types and scopes, and the code generator uses stored offsets to emit addresses. I'd implement it as a hash table for average $O(1)$ insert and lookup, and handle nested scopes with a stack of tables - push on entering a block, pop on leaving - so lookup goes from the innermost scope outward. That scope handling is exactly what makes shadowing work and lets the compiler catch undeclared variables, redeclarations, and type mismatches."

12. Quick Revision Notes

- Symbol table = compiler's database of identifiers and attributes (type, scope, offset, kind, signature).
- Operations: **insert** (on declaration), **lookup** (on use), enter/exit scope.
- Best structure: **hash table** ($O(1)$ avg); BST for ordering; list for tiny tables.
- Nested scopes: **stack of tables**; lookup inner $\rightarrow$ outer.
- Used by **all phases** + linker (cross-file symbol resolution).
- Catches: undeclared variable, redeclaration, type mismatch, wrong arg count.
- **Trap:** shadowing (legal, different scopes) $\neq$ redeclaration (error, same scope).
- Static scoping = by structure; dynamic = by call stack.

13. Practice Tasks

1. Design a symbol-table entry struct with fields for name, type, scope level, offset, and kind.
2. Implement `insert/lookup` with a hash map in C++/Python; add redeclaration detection.
3. Add scope handling with a stack of maps; test shadowing: global `x`, local `x`, verify inner wins.
4. Trace the symbol table through the nested-scope example in section 2 line by line.
5. Extend it to support function overloading by keying on name + parameter types.
6. Inspect real symbols: run `nm a.out` on a compiled binary and match entries to your source names.

14. Final Cheat Sheet

- **Core definition:** A compiler data structure mapping each identifier to its attributes (type, scope, offset, kind), used across all phases.
- **Why it matters:** Enables type checking, scope resolution, and address generation; source of most compile-time errors.
- **Most asked:** What does an entry store; best data structure (hash table); how are scopes handled; shadowing vs redeclaration; which phases use it.
- **Common comparisons:** Hash vs BST vs list; shadowing vs redeclaration; static vs dynamic scoping.

- **One-line answer:** "The symbol table is the compiler's name database - it maps every identifier to its type, scope, and memory location, and every phase from semantic analysis to code generation relies on it."

6. Error Handling

1. Overview

Definition. Error handling in a compiler is how it *detects*, *reports*, and *recovers from* mistakes in the source program so it can (ideally) continue and find more errors in a single run, rather than stopping at the first one. It is a cross-cutting responsibility of every phase.

Why it matters.

- Good diagnostics (clear message, correct location, helpful suggestion) hugely affect developer productivity.
- **Error recovery** lets the compiler report many errors at once instead of one-at-a-time, saving edit-compile cycles.
- Robustness: the compiler must never crash on bad input; it must fail gracefully.

Where it is used in real systems. Compiler diagnostics (Clang and Rust are famous for high-quality errors with suggestions), IDE red squiggles, linters, SQL parsers ("syntax error near ..."), and JSON/config parsers.

Why interviewers ask about it. It reveals whether you know *which phase catches which error* and whether you understand *error recovery strategies* (panic mode, phrase-level, etc.). It also ties into the difference between compile-time and runtime errors.

2. Core Idea

Errors are classified by the **phase** that detects them, and each detected error triggers a **recovery strategy** so compilation can proceed. The two goals are: (1) accurate detection and clear reporting, and (2) sensible recovery that avoids a cascade of spurious follow-on errors.

Intuition. A good proofreader does not stop at the first typo. They mark it, keep reading, and hand back the whole page with every mistake circled - without inventing fake errors caused by their own confusion after the first one.

Real-world analogy - grading an exam. A strict grader stops at the first wrong answer (like "stop on first error"). A helpful grader marks every mistake in one pass (error recovery), but must be careful: one early misread should not make them wrongly mark everything after it (cascading errors).

Small example.

```
int main() {  
    int x = ;           // syntax error: missing expression  
    y = x + 1;          // semantic error: y undeclared  
}
```

```
    return "hello";    // semantic error: returning string as int
}
```

A compiler with recovery reports **all three** errors with line numbers in one run, instead of only the first.

Types of errors by phase:

- **Lexical:** illegal character, malformed token (`123abc`, unterminated string).
- **Syntax:** grammar violation (missing `;`, unbalanced `{}`, `if` (`x` with no `)`).
- **Semantic:** type mismatch, undeclared/undefined name, wrong argument count.
- **Logical (not caught by compiler):** wrong algorithm, infinite loop - program compiles but behaves wrong.
- **Runtime (not a compile phase):** divide by zero, null dereference, array out of bounds.

3. Important Subtopics

3.1 Error classification by phase

- **What:** Which phase detects which error (lexical/syntax/semantic/runtime/logical).
- **Why it matters:** Explains error messages and where to look; a top interview discriminator.
- **Example:** `@` → lexical; missing `;` → syntax; undeclared var → semantic.
- **Interview angle:** Map a given error to its phase.

3.2 Panic-mode recovery

- **What:** On an error, skip input tokens until a designated **synchronizing token** (like `;`, `}`, or a keyword) is found, then resume.
- **Why it matters:** Simplest and most popular strategy; guarantees progress and avoids infinite loops.
- **Example:** After a bad statement, discard tokens up to the next `;` and continue.
- **Interview angle:** Trade-off - may skip real code and miss errors in the skipped region, but never loops forever.

3.3 Phrase-level recovery

- **What:** The parser performs a local correction (insert/delete/replace a token) to repair the input and continue, e.g. insert a missing `;`.
- **Why it matters:** More precise than panic mode; can produce better messages.
- **Example:** "expected `;`, inserted it" then keep parsing.
- **Interview angle:** Risk of incorrect repair leading to cascading errors.

3.4 Error productions

- **What:** The grammar is augmented with rules that explicitly match *common mistakes*, so the parser recognizes them and emits a specific, helpful message.
- **Why it matters:** Turns frequent errors into precise diagnostics ("did you mean `==` instead of `=?`").
- **Interview angle:** Requires anticipating common errors; used by production compilers.

3.5 Global correction

- **What:** Theoretically find the minimum number of changes to make the program valid (least-cost correction).
- **Why it matters:** Optimal in principle but too expensive in practice; mostly of academic interest.
- **Interview angle:** Know it exists and why it is impractical (costly).

3.6 Compile-time vs runtime errors

- **What:** Compile-time errors are caught before running (lexical/syntax/semantic); runtime errors happen during execution (divide by zero, null deref).
- **Why it matters:** A frequent point of confusion; runtime errors are handled by exceptions/traps, not compiler phases.

4. Real-World Example

The Rust compiler's diagnostics. When you write `let x = 5; x = 6;` Rust does not just say "error". It reports: the exact span underlined, the message "cannot assign twice to immutable variable `x`", a note pointing at the original binding, and a suggestion: "consider making this binding mutable: `let mut x`". This is production-grade error handling: precise *location* (from tracking source positions in tokens/AST), accurate *classification* (a borrow/mutability semantic check), and actionable *recovery advice*. Rust also continues after errors to report several at once. The same philosophy shows up in Clang's caret diagnostics and TypeScript's "Did you mean...?" suggestions - all built on tracking positions through every phase and applying recovery so one file yields many useful messages.

5. Diagrams / Mental Models

Error type by WHEN it is caught:

COMPILE TIME

Lexical bad char	Syntax missing;	Semantic type/scope
---------------------	--------------------	------------------------

detected & reported by phases

RUN TIME

divide by zero, null deref, out of bounds
--

traps/exceptions

Logical errors: compile AND run fine, but produce WRONG results (not caught).

Panic-mode recovery:

... bad token `X` $\rightarrow$ skip $\rightarrow$ skip $\rightarrow$ skip $\rightarrow$ `';` (sync token) $\rightarrow$ resume parsing

Recovery strategy	Idea	Pro	Con
Panic mode	skip to sync token	simple, no infinite loop	may skip valid code

Recovery strategy	Idea	Pro	Con
Phrase-level	local insert/delete/replace	targeted fix, better messages	wrong guess → cascades
Error productions	grammar rules for common errors	precise diagnostics	must anticipate errors
Global correction	least-cost fix	optimal	too expensive

6. Common Interview Questions

Q1. What is error handling in a compiler and what are its goals? Answer: Detecting, reporting, and recovering from source errors; goals are accurate reporting with good location/message and recovery so multiple errors are found in one run without crashing. Common mistake: mentioning only detection, not recovery.

Q2. Classify errors by the phase that detects them. Answer: Lexical (illegal characters/tokens), syntax (grammar violations), semantic (type/scope/undeclared), and runtime (not a compile phase). Logical errors are not caught by the compiler.

Q3. Which phase catches a missing semicolon vs an undeclared variable vs an illegal character? Answer: Missing `;` → syntax; undeclared variable → semantic; illegal character → lexical.

Q4. What is panic-mode recovery? Answer: On error, discard input tokens until a synchronizing token (like `;` or `}`) is reached, then resume parsing. Simple and loop-free but may skip valid code.

Q5. What is phrase-level recovery? Answer: Making a small local correction (insert/delete/replace a token) to repair the input and continue, e.g. inserting a missing semicolon.

Q6. Why is error recovery important? Answer: So the compiler reports many errors in a single compilation instead of stopping at the first, reducing edit-compile cycles.

Q7. What is the difference between compile-time and runtime errors? Answer: Compile-time errors (lexical/syntax/semantic) are caught before execution; runtime errors (divide by zero, null deref) occur during execution and are handled by exceptions/traps.

Q8. What are error productions? Answer: Extra grammar rules that match common mistakes so the parser recognizes them and emits a specific, helpful message.

Q9. What are cascading errors and how do you avoid them? Answer: Spurious errors caused by a bad recovery after a real error. Avoided by good synchronization (sensible sync tokens) and suppressing follow-on errors near a recovery point.

Q10. Can a program compile with no errors but still be wrong? Answer: Yes - logical errors (wrong algorithm) pass all compile checks but produce incorrect results.

Q11. Why must a compiler not stop at the first error? Answer: To be productive: reporting all errors at once lets the developer fix many issues per compile instead of one.

7. Deep-Dive Questions

D1. How does the compiler know the exact line and column of an error? Each token carries source position information (line, column, byte offset) attached by the lexer; the parser and semantic analyzer propagate these into AST nodes, so any phase can report the precise span. Good diagnostics come from faithfully threading positions through all phases.

D2. Why can recovery cause cascading (spurious) errors, and what mitigates it? After an error the compiler's internal state (parse stack, symbol table) may be inconsistent, so subsequent valid code looks wrong. Mitigations: choose robust synchronizing tokens, suppress additional errors within a small window after a report, and insert "error" placeholder nodes so later phases skip already-broken subtrees.

D3. How are semantic errors detected differently from syntax errors? Syntax errors come from the grammar (context-free) and are found by the parser. Semantic errors need *context* (types, declarations) that a CFG cannot express, so they are found by the semantic analyzer using the symbol table - e.g. "variable used before declaration" is not a grammar violation.

D4. How do statically typed languages catch more errors than dynamically typed ones at compile time? Static type systems require types to be known and checked at compile time, so type mismatches, wrong arguments, and undefined-method calls are caught before running. Dynamic languages defer these to runtime, so the "same" bug surfaces only when that line executes (or never, if untested).

D5. What is the difference between an error, a warning, and a note? An **error** prevents successful compilation (invalid program). A **warning** flags suspicious-but-legal code (unused variable, implicit conversion) without failing the build. A **note** adds context to an error/warning (e.g. "previous declaration was here"). Treating warnings as errors (`-Werror`) enforces stricter quality.

8. Comparison Tables

Error type by phase

Error kind	Example	Detected in	Caught before running?
Lexical	@, 123abc, unterminated string	Lexer	Yes
Syntax	missing ;, unbalanced {}	Parser	Yes
Semantic	type mismatch, undeclared var	Semantic analyzer	Yes
Runtime	divide by zero, null deref	Execution	No
Logical	wrong formula, off-by-one	Nowhere (compiles)	No

Recovery strategies

Strategy	Precision	Cost	Risk
Panic mode	low	very low	skips code
Phrase-level	medium	low	wrong repair
Error productions	high (for anticipated errors)	medium	limited to known cases
Global correction	highest	very high	impractical

Error vs Warning

	Error	Warning
Program validity	invalid	valid but suspicious
Build result	fails	succeeds
Example	type mismatch	unused variable

9. Common Mistakes

- Confusing which phase catches which error (missing `;` is syntax, not lexical; type mismatch is semantic, not syntax).
- Thinking a compiler stops at the first error (good compilers recover and continue).
- Treating runtime errors as compile-time phases.
- Believing "compiles cleanly" means "correct" (logical errors survive).
- Ignoring cascading errors as a real problem of recovery.
- Confusing warnings with errors.

10. Edge Cases / Special Cases

- **Unterminated string/comment** is a lexical error that can swallow the rest of the file, producing misleading later errors.
- **Missing closing brace** can make the parser blame a line far from the real mistake.
- **Cascading errors**: one real error can generate dozens of fake ones; experienced developers fix the first and recompile.
- **Ambiguous grammars** can make it unclear which recovery is correct.
- **Warnings-as-errors** can block builds on benign code; teams tune this deliberately.
- **Runtime vs compile-time division**: `1/0` with constant operands may be caught at compile time (constant folding) but `a/b` cannot.

11. How to Explain in Interview

"Compiler error handling has three jobs: detect, report, and recover. Errors are classified by the phase that finds them - lexical for bad characters, syntax for grammar violations like a missing semicolon, and semantic for things like type mismatches or undeclared variables, all using the symbol table. Runtime errors like divide-by-zero are separate; they happen during execution, not in a compile phase. The important design goal is recovery: instead of stopping at the first error, the compiler synchronizes - typically panic mode, skipping to the next semicolon or brace - so it can report many errors in one run. The tricky part is avoiding cascading spurious errors, which good synchronization and precise source positions help prevent."

12. Quick Revision Notes

- Three jobs: **detect**, **report**, **recover**.
- By phase: **lexical** (bad char), **syntax** (missing `;/}`), **semantic** (type/undeclared), **runtime** (divide by zero - not a phase), **logical** (wrong result - uncaught).
- Recovery strategies: **panic mode** (skip to sync token), **phrase-level** (local fix), **error productions** (grammar rules for common mistakes), **global correction** (optimal, impractical).

- Goal of recovery: report **many errors per run**, never crash.
- Cascading errors = spurious follow-ons; mitigate with good sync tokens.
- **Trap:** "compiles" != "correct" (logical errors). Runtime error != compile phase.
- Error (fails build) vs warning (suspicious but legal).

13. Practice Tasks

1. For a snippet with a lexical, a syntax, and a semantic error, label each error's phase.
2. Introduce a missing `;` in C and observe how far off the reported line is; explain why.
3. Implement panic-mode recovery in a tiny recursive-descent parser: skip to the next `;` on error.
4. Trigger cascading errors by removing a `{`; count spurious follow-on messages.
5. Compile the same bug in a statically typed (C) and dynamically typed (Python) language; note when it is caught.
6. Add an "error production" so `if x > 0` (missing parentheses) yields a helpful custom message.

14. Final Cheat Sheet

- **Core definition:** Detecting, reporting, and recovering from source errors across all phases so multiple errors are found per run.
- **Why it matters:** Developer productivity, robustness, and clear diagnostics.
- **Most asked:** Classify errors by phase; panic-mode vs phrase-level recovery; compile-time vs runtime; why not stop at first error.
- **Common comparisons:** Lexical vs syntax vs semantic vs runtime; recovery strategies; error vs warning.
- **One-line answer:** "Error handling detects, reports, and recovers from source errors phase by phase - lexical, syntax, semantic - using recovery like panic mode so the compiler reports many errors in one run instead of dying on the first."

7. One-Pass vs Multi-Pass Compiler

1. Overview

Definition. A **pass** is one complete scan over the program (source or an intermediate representation). A **one-pass compiler** translates the source in a single traversal, generating target code as it parses. A **multi-pass compiler** traverses the program (or its IR) several times, each pass doing a different job (parse, then type-check, then optimize, then generate code).

Why it matters.

- It affects **compilation speed vs. optimization quality, memory usage**, and **language feature support** (forward references, whole-program optimization).
- It explains why old languages (Pascal) were designed for one-pass compilation and why modern optimizing compilers are multi-pass.

Where it is used in real systems.

- **One-pass (or few-pass):** classic Pascal and early C compilers, some Wirth-family languages, simple scripting translators - prioritized speed and low memory.

- **Multi-pass:** GCC, Clang/LLVM, `javac` + JVM, virtually all optimizing compilers - prioritize optimization and features.

Why interviewers ask about it. It tests understanding of trade-offs (speed vs optimization), why forward references force multiple passes or forward declarations, and how the phase structure maps onto passes.

2. Core Idea

The question is: **how many times do we walk the program?** Some work can be done "on the fly" in a single walk; other work (global optimization, resolving names defined later) inherently needs to see more of the program before acting, forcing additional passes.

Intuition. Reading a document once and acting immediately is fast but shallow - you cannot use information that appears later. Reading it several times lets each pass build on a full picture from the previous one, enabling deeper analysis.

Real-world analogy - editing an essay. A one-pass editor fixes each sentence as they read it, top to bottom, once. A multi-pass editor reads once for structure, again for grammar, again for style, again for consistency. The multi-pass editor produces a far more polished result but spends more time; the one-pass editor is quick but cannot fix a paragraph based on something written three pages later.

Small example - forward reference.

```
void a() { b(); }    // b used here...
void b() { ... }    // ...but defined later
```

A strict one-pass compiler processing top to bottom hits `b()` before `b` is declared. It must either require a **forward declaration** (`void b();` up top) or make **two passes** (pass 1: collect all declarations into the symbol table; pass 2: generate code with all names known).

Why multiple passes enable optimization. Optimizations like global common-subexpression elimination or inlining need to analyze the *whole* function/program first, then transform - that is naturally two logical passes (analyze, then rewrite).

3. Important Subtopics

3.1 One-pass compilation

- **What:** Lexing, parsing, semantic checks, and code generation interleaved in a single traversal.
- **Why it matters:** Very fast and low memory; historically important for limited hardware.
- **Example:** Turbo Pascal was famous for near-instant one-pass compilation.
- **Interview angle:** Limitations - little/no global optimization; forward references need forward declarations.

3.2 Multi-pass compilation

- **What:** Several traversals, each a distinct stage (build AST, semantic analysis, optimization, code generation), passing an IR between passes.
- **Why it matters:** Enables strong optimization, cleaner modular design, and full language features.

- **Example:** GCC/LLVM run dozens of optimization passes over the IR.
- **Interview angle:** Slower and more memory, but better code and portability (IR reuse).

3.3 Forward references

- **What:** Using a name (function, type, variable) before it is textually declared.
- **Why it matters:** The classic reason a language needs multiple passes or forward declarations.
- **Example:** Mutually recursive functions; using a class defined later in the file.
- **Interview angle:** C's header/prototype requirement is a one-pass-friendly design choice.

3.4 Pass vs Phase (do not confuse them)

- **What:** A **phase** is a logical stage of work (lexing, parsing...). A **pass** is one physical traversal of the program. One pass can perform several phases; one phase can span multiple passes.
- **Why it matters:** A very common interview confusion.
- **Interview angle:** "Is a phase the same as a pass?" - No.

3.5 Memory vs speed vs quality trade-off

- **What:** One-pass = fast, low memory, weaker code. Multi-pass = slower, more memory, better optimized code.
- **Why it matters:** Explains real compiler design decisions and flags (`-O0` vs `-O2` runs more passes).

4. Real-World Example

GCC/Clang at different optimization levels. When you compile with `-O0`, the compiler does close to the minimum work - roughly parse, minimal analysis, and generate code - so builds are fast for development. When you compile with `-O2` or `-O3`, the compiler runs *many additional passes* over the intermediate representation: inlining, loop unrolling, vectorization, dead-code elimination, common-subexpression elimination, and more, each a separate traversal that analyzes then transforms the IR. That is why a release build is slower to compile but produces faster binaries. This is the one-pass vs multi-pass trade-off exposed as a knob: more passes cost compile time and memory but yield better machine code. Contrast this with historical Turbo Pascal, whose one-pass design made it feel instant on 1980s hardware but limited optimization.

5. Diagrams / Mental Models

ONE-PASS:

```
source → [ lex + parse + check + codegen, all in ONE traversal ] → target
(fast, low memory, forward refs need forward declarations)
```

MULTI-PASS:

```
source → [Pass 1: build AST] → [Pass 2: semantic/type check]
      → [Pass 3: optimize IR] → ... → [Pass N: codegen] → target
(slower, more memory, strong optimization, full features)
```

Dimension	One-pass	Multi-pass
Traversals	1	many
Speed	fast	slower
Memory	low	higher (holds IR/AST)
Optimization	minimal	extensive
Forward references	need forward declaration	handled naturally
Design	simpler, coupled	modular, IR-based
Examples	classic Pascal, early C	GCC, LLVM, javac

6. Common Interview Questions

Q1. What is a pass in a compiler? Answer: One complete traversal over the source or an intermediate representation. Common mistake: equating a pass with a phase.

Q2. Difference between one-pass and multi-pass compilers? Answer: One-pass translates in a single traversal generating code on the fly; multi-pass makes several traversals, each doing a distinct job, enabling better optimization and feature support at the cost of speed and memory.

Q3. What is the difference between a pass and a phase? Answer: A phase is a logical stage (lexing, parsing, etc.); a pass is a physical traversal. One pass may run several phases, and one phase may take multiple passes.

Q4. Why might a language require multiple passes? Answer: Forward references (using names defined later), whole-program optimization, and separating analysis from transformation all need more than one traversal.

Q5. How does a one-pass compiler handle forward references? Answer: It requires forward declarations/prototypes (like C function prototypes) so the name is known when first used.

Q6. What are the advantages of one-pass compilers? Answer: Faster compilation, lower memory usage, simpler implementation.

Q7. What are the advantages of multi-pass compilers? Answer: Strong optimization, support for forward references and complex features, modular design, and IR reuse across languages/targets.

Q8. Why do optimizing compilers need multiple passes? Answer: Optimizations must first analyze the whole function/program, then transform it - inherently more than one traversal; many optimizations also enable further ones, requiring repeated passes.

Q9. Which is more memory-intensive and why? Answer: Multi-pass, because it must retain the AST/IR between passes rather than discarding code after emitting it.

Q10. Give a real example of the trade-off in practice. Answer: `gcc -O0` (few passes, fast build) vs `gcc -O2` (many passes, slower build, faster binary).

7. Deep-Dive Questions

D1. Can a purely one-pass compiler ever do global optimization? Why or why not? Not truly, because global optimization requires knowledge of code that appears later, which a single forward traversal has not seen when it emits code. It can do only *local* (peephole/basic-block) optimizations on what it has already processed. Global optimization inherently needs a second analysis pass over the whole unit.

D2. Why was Pascal designed to be one-pass-friendly while C uses header files? Both target fast compilation on limited hardware. Pascal requires declarations before use and uses **forward** for mutual recursion so a single pass suffices. C uses prototypes/headers so the compiler knows a function's signature before its definition is seen, again enabling essentially one-pass translation per file.

D3. How does multi-pass design enable the front-end/back-end split and IR reuse? Because passes communicate through a well-defined IR, the front end (source → IR) and back end (IR → machine code) become separate pass groups. This lets many languages share optimization/back-end passes and lets one compiler target many machines - the **N+M** architecture depends on multi-pass, IR-based design.

D4. Do interpreters have passes? Modern interpreters typically do at least one pass to compile source to bytecode (a compile pass) and then execute it; some do multiple analysis passes (Python builds an AST then compiles to bytecode). So the pass concept applies to interpreter front ends too.

D5. How do you decide how many passes to run? It is a cost/benefit decision: each additional optimization pass costs compile time and memory but may improve the output. Compilers expose this via **-O** levels and may iterate certain passes until no further changes occur (a fixpoint), stopping when marginal gains are small.

8. Comparison Tables

One-pass vs Multi-pass

Feature	One-pass	Multi-pass
Number of traversals	1	2+
Compilation speed	fast	slower
Memory usage	low	higher
Optimization quality	minimal (local only)	high (global)
Forward references	forward declarations required	naturally supported
Implementation	simpler, coupled	modular, IR-based
Portability (IR reuse)	low	high
Examples	Turbo Pascal, early C	GCC, Clang/LLVM, javac

Pass vs Phase

	Pass	Phase
Meaning	one physical traversal	one logical stage of work
Example	"the optimization pass over IR"	"semantic analysis"
Relationship	a pass can run several phases	a phase can span several passes

9. Common Mistakes

- Confusing a **pass** (traversal) with a **phase** (logical stage).
- Thinking one-pass compilers cannot compile real languages (they can, with forward declarations).
- Believing more passes are always better (they cost time/memory; there is a trade-off).
- Assuming multi-pass means "slow to run" (it is slow to *compile*; the produced program is usually faster).
- Forgetting that global optimization fundamentally needs more than one pass.
- Thinking C needs headers due to a language flaw rather than a one-pass-friendly design.

10. Edge Cases / Special Cases

- **Mutually recursive functions/types** force forward declarations in one-pass languages.
- **Two-pass assemblers** are the classic example: pass 1 builds the symbol table of labels, pass 2 resolves addresses (forward jumps).
- **Backpatching** lets a near-one-pass code generator emit jumps with blank targets and fill them in later, reducing passes.
- **Iterative optimization passes** run until a fixpoint (no more changes), so "number of passes" can be dynamic.
- **Separate compilation** (per-file) is roughly one-pass per file, but linking is another whole stage.
- **Link-time optimization (LTO)** adds a whole-program pass at link time for cross-file optimization.

11. How to Explain in Interview

"A pass is one complete traversal of the program. A one-pass compiler does everything - parse, check, generate code - in a single walk, so it is fast and low-memory but can only do local optimization and needs forward declarations for names used before they are defined, like C prototypes. A multi-pass compiler walks the program several times, each pass doing a distinct job through an intermediate representation, which lets it do global optimization, support forward references naturally, and reuse passes across languages and targets. The trade-off is compile speed and memory versus code quality - that is literally what `-O0` versus `-O2` in GCC toggles. And one subtlety: a pass is not the same as a phase; a single pass can run several phases."

12. Quick Revision Notes

- **Pass** = one full traversal; **phase** = one logical stage (not the same).
- One-pass: single traversal, fast, low memory, local optimization only, needs forward declarations.
- Multi-pass: many traversals via IR, slower, more memory, global optimization, full features, portable.
- Forward reference = using a name before it is declared → needs forward declaration or extra pass.
- `gcc -O0` (few passes, fast build) vs `-O2` (many passes, faster binary).
- Two-pass assembler: pass 1 = symbol/label table, pass 2 = resolve addresses.
- **Trap:** more passes ≠ always better; multi-pass = slow *compile*, fast *program*.

13. Practice Tasks

1. Write two mutually recursive functions in C without prototypes and observe the compile error; fix with a forward declaration.
2. Compile a program with `-O0` and `-O2`; compare compile time and the generated assembly size/speed.
3. Hand-simulate a two-pass assembler on code with a forward jump: build the label table (pass 1), then resolve (pass 2).

4. List which phases a minimal one-pass compiler could combine into a single traversal.
5. Explain why global common-subexpression elimination cannot be done in a single forward pass.
6. Sketch the pass pipeline of an optimizing compiler from source to machine code, labeling each pass's job.

14. Final Cheat Sheet

- **Core definition:** One-pass = translate in a single traversal; multi-pass = several traversals, each a distinct job via an IR.
 - **Why it matters:** Trades compile speed/memory against optimization quality and feature support.
 - **Most asked:** Pass vs phase; why multiple passes (forward refs, optimization); how one-pass handles forward references; advantages of each.
 - **Common comparisons:** One-pass vs multi-pass; pass vs phase; **-O0** vs **-O2**.
 - **One-line answer:** "A one-pass compiler translates in a single fast, low-memory traversal but does only local optimization and needs forward declarations; a multi-pass compiler walks the program several times through an IR to enable global optimization and full language features at the cost of compile time and memory."
-

8. Just-In-Time (JIT) Compilation

1. Overview

Definition. Just-In-Time (JIT) compilation is compiling code to native machine code *at runtime*, just before (or while) it executes, rather than entirely ahead of time. A JIT typically starts by interpreting bytecode, watches which parts run frequently ("hot" code), and then compiles those hot parts to optimized native code on the fly, often using **runtime profiling** to optimize better than a static compiler could.

Why it matters.

- It combines an interpreter's fast startup and portability with a compiler's fast steady-state execution.
- It can exploit information only available at runtime (actual types, hot paths, branch behavior) to optimize better than ahead-of-time compilation in some cases.

Where it is used in real systems. The **JVM** (HotSpot's C1/C2 compilers), **.NET CLR**, **V8** (JavaScript, Chrome/Node.js - Ignition interpreter + TurboFan/Maglev JITs), **PyPy** (tracing JIT for Python), **LuaJIT**, and modern browser JS engines.

Why interviewers ask about it. It ties together compiler vs interpreter, the hybrid model, profiling, and performance trade-offs (warm-up, memory). It is central to how Java/JS/C# actually run and a favorite for "how does the JVM run fast" questions.

2. Core Idea

Compile lazily and adaptively. Do not pay to compile everything upfront; interpret first for instant startup, measure what is actually hot, and spend compilation effort only where it pays off - using real runtime data to guide optimization.

Intuition. Most programs spend most of their time in a small fraction of code (the 90/10 rule). A JIT identifies that hot 10% and turns just it into fast native code, while cold code stays interpreted. Because it watches the program run, it knows things a static compiler can only guess.

Real-world analogy - a smart tour guide. On your first visit, the guide explains each spot on the spot (interpreting). Noticing you keep returning to the same museum room, the guide prepares a fast, polished, tailored tour of *that* room (JIT-compiles the hot path), optimized for exactly how you use it. Rooms you visit once are never worth the prep.

Small example (JVM tiered flow).

```
method run 1..N times    → interpreted (fast startup, collects profile)
method becomes "hot"     → C1 compiler: quick native code with basic optimization
method becomes "hotter" → C2 compiler: heavily optimized native code using profile
assumption violated      → deoptimize: fall back to interpreter, recompile later
```

Speculative optimization + deoptimization. A JIT may *assume* a call site always sees the same object type (monomorphic) and inline aggressively. If a different type shows up later, it **deoptimizes** - discards the specialized code and falls back to the interpreter, then may recompile with the new information. This "guess based on observed behavior, undo if wrong" ability is what makes JITs powerful and is impossible for pure AOT.

3. Important Subtopics

3.1 Hotspot detection / profiling

- **What:** Counting method invocations and loop iterations to find frequently executed ("hot") code.
- **Why it matters:** Focuses expensive compilation only where it pays off.
- **Example:** JVM compiles a method after it crosses an invocation/backedge threshold.
- **Interview angle:** "How does a JIT decide what to compile?" - runtime profiling counters.

3.2 Tiered compilation

- **What:** Multiple compilers of increasing optimization: interpret → quick JIT (C1) → optimizing JIT (C2).
- **Why it matters:** Balances fast warm-up (quick tier) against peak performance (optimizing tier).
- **Example:** JVM tiers; V8's Ignition → Sparkplug → Maglev → TurboFan.
- **Interview angle:** Why multiple tiers? Quick code sooner, best code where it matters.

3.3 Speculative optimization and deoptimization

- **What:** Optimizing based on runtime assumptions (type stability, unlikely branches), with the ability to bail out if the assumption breaks.
- **Why it matters:** Enables aggressive optimizations (inlining virtual calls) that static compilers cannot do safely.
- **Example:** Inlining a monomorphic call; deoptimizing when a new type appears.
- **Interview angle:** This is a JIT's unique superpower - profile-guided, undoable optimization.

3.4 Warm-up cost

- **What:** The initial period where code is interpreted and being profiled/compiled, so performance is below peak.
- **Why it matters:** Short-lived programs may never reach peak speed; a real downside vs AOT.
- **Interview angle:** "Why can a JIT be slow at start?" - warm-up + compilation overhead.

3.5 Runtime cost (memory and CPU)

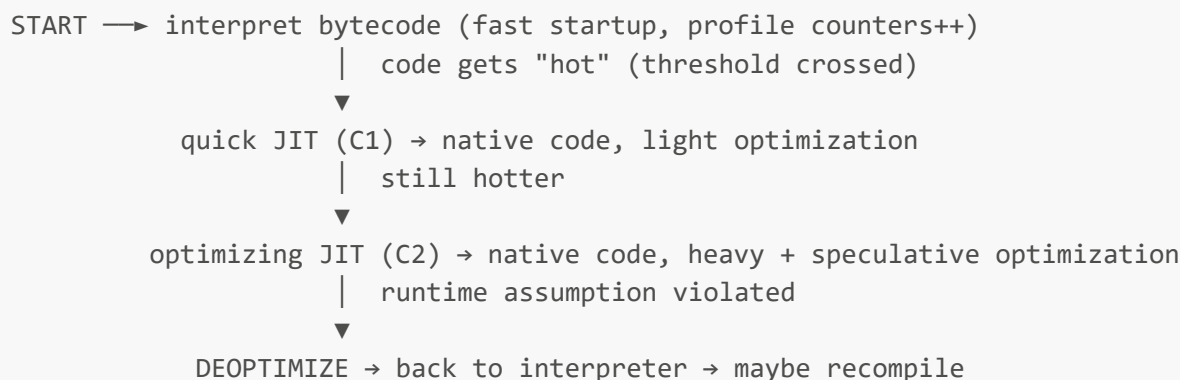
- **What:** The compiler itself runs in the same process, consuming memory and CPU while the app runs.
- **Why it matters:** JITs have higher memory footprints; unsuitable for very constrained environments.
- **Interview angle:** Trade-off vs AOT's zero runtime compilation cost.

4. Real-World Example

V8 running JavaScript in Chrome and Node.js. When a webpage's JS loads, V8 must react instantly, so it parses to bytecode and runs it in the **Ignition** interpreter - no waiting for compilation, the page is interactive immediately. As the user interacts, hot functions (an animation loop, a data-processing routine) cross V8's thresholds and are handed to optimizing JITs (**Maglev** then **TurboFan**), which generate fast native code specialized to the *types actually observed* (for example assuming an array holds only integers). If later a string sneaks into that array, V8 **deoptimizes** the function back to bytecode and may recompile. This is why modern JavaScript can start instantly yet run numeric loops at near-native speed - impossible with a pure interpreter (too slow at steady state) or pure AOT (cannot specialize to runtime types or start as fast).

5. Diagrams / Mental Models

JIT execution timeline:



Performance over time: low (warm-up) —rising—→ HIGH (steady state)

Concept	Meaning
Hotspot	frequently executed code worth compiling
Tier	a compiler level (interpret / quick / optimizing)
Speculation	optimize on a runtime assumption
Deoptimization	undo specialized code when assumption fails
Warm-up	time to reach peak performance

6. Common Interview Questions

Q1. What is JIT compilation? Answer: Compiling code to native machine code at runtime, just before/while it executes, typically after interpreting and profiling to find hot code, using runtime information to optimize. Common mistake: describing it as a fancy interpreter (it is a runtime *compiler*).

Q2. How is JIT different from AOT compilation? Answer: AOT compiles before execution once; JIT compiles during execution, adaptively, using runtime profiles - trading startup/warm-up and memory for potentially better, data-driven optimization.

Q3. Why does a JIT interpret first instead of compiling everything immediately? Answer: Fast startup, and to gather profiling data so it compiles only hot code and optimizes it based on real behavior, avoiding wasted compilation of rarely run code.

Q4. What is hotspot detection? Answer: Using runtime counters (invocations, loop back-edges) to identify frequently executed code that is worth compiling.

Q5. What is tiered compilation? Answer: Using multiple compilers of increasing optimization (interpret → quick JIT → optimizing JIT) to balance fast warm-up with peak performance.

Q6. What is deoptimization? Answer: Discarding speculatively optimized native code and falling back to the interpreter when a runtime assumption (like a stable type) is violated, then possibly recompiling.

Q7. What is the warm-up problem? Answer: Early execution is interpreted and being profiled/compiled, so performance is below peak; short-lived programs may never reach top speed.

Q8. Can a JIT be faster than an AOT compiler? How? Answer: Yes, sometimes - it can use runtime information (actual types, hot paths, branch probabilities) to specialize and inline in ways a static compiler cannot safely assume.

Q9. What are the downsides of JIT? Answer: Slower startup/warm-up, higher memory and CPU usage (the compiler runs in-process), unpredictable pauses during compilation, and complexity.

Q10. Give three systems that use JIT. Answer: JVM (HotSpot), .NET CLR, and V8 (also PyPy, LuaJIT).

Q11. Does JIT require bytecode? Answer: Not strictly, but it is usual - JITs typically compile a portable bytecode/IR to native code; some JIT directly from source or an AST.

7. Deep-Dive Questions

D1. How can a JIT inline a virtual/polymorphic method call when the target is not known statically? It observes at runtime that the call site is monomorphic (always the same concrete type) and speculatively inlines that implementation, guarding with a type check. If the guard fails (a different type arrives), it deoptimizes. Static AOT compilers usually cannot do this because they must be correct for all possible types without runtime evidence.

D2. Why does a JIT need a deoptimization mechanism at all? Because its optimizations are based on *assumptions* that could later be false (a class is loaded that overrides a method, a variable's type changes). To stay correct, it must be able to abandon the optimized code and resume in a safe, general interpreter state precisely mapped from the optimized state (via "deopt points" / on-stack replacement metadata).

D3. What is On-Stack Replacement (OSR)? OSR swaps a currently-running method's execution from interpreted to compiled code (or vice versa for deopt) *while it is on the stack* - important for long-running loops that become hot mid-execution, so the running loop can jump to native code without waiting for the next call.

D4. When would you disable or avoid JIT? For very short-lived processes (CLI tools, serverless cold starts) where warm-up never amortizes; memory-constrained or real-time systems where the compiler's footprint and unpredictable pauses are unacceptable; or where startup latency and determinism matter more than peak throughput - hence AOT options like GraalVM native image.

D5. How does garbage collection interact with JIT? JIT-compiled code must cooperate with the GC: it records where object references live (stack maps) at safepoints so the GC can find and update pointers, and moving GCs may require the JIT to insert read/write barriers. This coupling adds complexity that AOT runtimes also share but must be baked into generated native code.

8. Comparison Tables

JIT vs Interpreter vs AOT

Feature	Interpreter	JIT	AOT
When compiled	never (executes directly)	at runtime, hot code	before runtime
Startup	fast	fast (interprets first)	fast (already native)
Peak speed	low	high (after warm-up)	high
Uses runtime profile	no	yes	no
Memory/CPU at runtime	low-medium	high (compiler in-process)	low
Portability of artifact	high	high (bytecode)	low (native)
Examples	Bash	JVM, V8, .NET	C, Rust, Go

JIT internals

Term	Role
Interpreter tier	instant startup + profiling
Quick JIT (C1)	fast native code, light optimization
Optimizing JIT (C2/TurboFan)	best native code, speculative optimization
Deoptimization	correctness fallback when speculation fails

9. Common Mistakes

- Calling JIT "a smarter interpreter" - it is a runtime *compiler*.
- Thinking JIT compiles everything at startup (it interprets first, compiles hot code lazily).
- Ignoring warm-up and memory costs (JIT is not free).
- Believing AOT is always faster (JITs can beat AOT via profile-guided specialization).
- Forgetting deoptimization exists and why (correctness of speculative optimization).

- Assuming JIT needs no bytecode/IR (usually it uses one).

10. Edge Cases / Special Cases

- **Cold start penalty:** serverless/CLI workloads may prefer AOT because JIT never warms up.
- **Deopt storms:** pathological type instability causes repeated deopt/recompile, hurting performance.
- **JIT + AOT hybrids:** JVM's Ahead-of-Time (jaotc/GraalVM) and .NET ReadyToRun precompile to reduce warm-up, then JIT further.
- **Tracing JITs** (PyPy, LuaJIT) compile hot *loop traces* rather than whole methods.
- **Security:** JITs need writable+executable memory (W^X concerns); "JIT spraying" is an attack class.
- **Meta-tracing / self-optimizing interpreters** (Truffle/GraalVM) generate JITs from interpreter definitions.

11. How to Explain in Interview

"JIT compilation compiles code to native machine code at runtime instead of ahead of time. The runtime starts by interpreting bytecode for instant startup while counting how often each method and loop runs. When something becomes hot, it compiles just that code to optimized native code, often using tiers - a quick compiler first, then a heavy optimizing one. The key advantage over static compilation is that it uses real runtime data: it can specialize on the actual types it sees and inline virtual calls, guarding with checks and deoptimizing back to the interpreter if an assumption breaks. The costs are warm-up time and higher memory, since the compiler runs inside the process. That is exactly how the JVM and V8 get fast startup and near-native steady-state speed at the same time."

12. Quick Revision Notes

- JIT = compile to native **at runtime**, after interpreting + profiling hot code.
- Flow: **interpret** → **detect hotspots** → **quick JIT** → **optimizing JIT** → **deoptimize if wrong**.
- **Tiered compilation** balances warm-up vs peak speed.
- **Speculative optimization** (guess on runtime types) + **deoptimization** (undo) = JIT's superpower.
- Pros: fast startup + fast steady state + profile-guided optimization. Cons: warm-up, memory/CPU, pauses.
- Users: JVM (HotSpot C1/C2), .NET CLR, V8, PyPy, LuaJIT.
- **Trap:** JIT is a compiler, not an interpreter; it can beat AOT using runtime data.

13. Practice Tasks

1. Run a JVM program with `-XX:+PrintCompilation` and watch methods get compiled as they become hot.
2. Compare a tight loop's first-iteration vs steady-state time in Java or Node to observe warm-up.
3. In Node.js, use `--print-opt-code / %OptimizeFunctionOnNextCall` (with `--allow-natives-syntax`) to see V8 optimize a function.
4. Force deoptimization: write a JS function optimized for numbers, then pass a string; observe the deopt.
5. Compare startup time of a Java "hello world" vs a Go (AOT) "hello world"; explain the difference.
6. Explain, for a program that runs for 50 ms vs 50 minutes, whether JIT or AOT is the better fit and why.

14. Final Cheat Sheet

- **Core definition:** Compiling code to native machine code at runtime, after interpreting and profiling, using runtime data to optimize hot code.
 - **Why it matters:** Fast startup + fast steady state; can out-optimize AOT via runtime specialization.
 - **Most asked:** JIT vs AOT; hotspot detection; tiered compilation; deoptimization; warm-up problem.
 - **Common comparisons:** Interpreter vs JIT vs AOT; quick tier vs optimizing tier.
 - **One-line answer:** "JIT compiles hot code to optimized native machine code at runtime using profiling and speculation, giving an interpreter's fast startup with a compiler's steady-state speed - at the cost of warm-up and memory."
-

9. Ahead-Of-Time (AOT) Compilation

1. Overview

Definition. Ahead-Of-Time (AOT) compilation translates source code (or bytecode) fully into native machine code *before* the program runs, producing a standalone executable. All compilation work is done at build time; nothing is compiled at runtime. This is the "classic" compilation model.

Why it matters.

- It gives **fast, predictable startup** (no warm-up), **no runtime compilation overhead**, **lower memory footprint**, and **deterministic performance** - crucial for CLI tools, embedded systems, and cold-start-sensitive serverless functions.
- It is the standard for systems languages (C, C++, Rust, Go) and increasingly used to precompile traditionally JIT'd languages (GraalVM native image for Java, .NET Native AOT).

Where it is used in real systems. C/C++/Rust/Go binaries, iOS apps (Swift is AOT-compiled - Apple disallows runtime JIT for App Store apps), Android's ART (AOT-compiles apps at install time), GraalVM native images, .NET Native AOT, embedded firmware.

Why interviewers ask about it. It is the natural counterpart to JIT. Interviewers want to see you articulate the AOT vs JIT trade-off precisely (startup and predictability vs runtime adaptivity) and know real scenarios where AOT wins.

2. Core Idea

Do all the work upfront. Compile and optimize the entire program before it ships, so at runtime there is nothing left to translate - the CPU just runs native instructions immediately.

Intuition. Prepare everything in advance so execution is instant and predictable. You trade away the ability to adapt to runtime data (which a JIT has) in exchange for zero runtime compilation cost and consistent performance.

Real-world analogy - a printed map vs live GPS. AOT is a fully printed map handed to you before the trip: instantly usable, no processing during the drive, but it cannot adapt to live traffic. JIT is live GPS: it adapts to current conditions but needs to keep computing while you drive. If you value instant, predictable, low-overhead navigation, you print the map ahead of time.

Small example.

```
Go build:  go build main.go  → single native binary (all compiled AOT)
Run:      ./main             → executes immediately, no compiler involved
```

The entire program - including its runtime and garbage collector for Go - is baked into the binary at build time. Startup is a few milliseconds because there is nothing to compile or warm up.

Why AOT cannot always match JIT's peak: it must be correct for *all* possible runtime inputs and types, so it cannot speculate on "this call site is always type X". It uses static analysis and optionally **Profile-Guided Optimization (PGO)** - feeding profiles from a training run back into the AOT build - to recover some of that advantage.

3. Important Subtopics

3.1 Build-time compilation and standalone binaries

- **What:** The full pipeline (front end, optimizer, back end) runs at build time, emitting a native executable.
- **Why it matters:** No runtime dependency on a compiler; simple deployment (ship one binary).
- **Example:** `gcc`, `rustc`, `go build`.
- **Interview angle:** Contrast with JIT needing the compiler present at runtime.

3.2 Fast, predictable startup (no warm-up)

- **What:** Native code runs at full speed from the first instruction; no interpret-then-compile ramp.
- **Why it matters:** Critical for short-lived processes, CLIs, and serverless cold starts.
- **Interview angle:** "Why is AOT better for a CLI tool?" - instant peak performance, no warm-up.

3.3 Lower memory footprint and determinism

- **What:** No in-process compiler, no profiling structures; performance does not fluctuate as a JIT recompiles.
- **Why it matters:** Suits embedded/real-time systems where memory is tight and pauses are unacceptable.
- **Interview angle:** Determinism matters for real-time guarantees.

3.4 Profile-Guided Optimization (PGO)

- **What:** Run an instrumented build on representative inputs, collect a profile, then recompile using it to guide inlining, branch layout, etc.
- **Why it matters:** Recovers some of JIT's data-driven advantage while staying AOT.
- **Example:** `gcc -fprofile-generate` then `-fprofile-use`; Go PGO.
- **Interview angle:** PGO is "static compilation with runtime hints from a training run".

3.5 Limitations vs JIT

- **What:** Cannot use *live* runtime data, cannot speculate/deoptimize, must handle all cases conservatively, and produces platform-specific binaries (needs cross-compilation for other targets).
- **Why it matters:** Explains why some workloads still prefer JIT.
- **Interview angle:** The AOT trade-off is predictability/startup vs runtime adaptivity.

4. Real-World Example

iOS apps and Android ART. Apple requires App Store apps to be AOT-compiled - Swift and Objective-C are compiled to native ARM machine code before shipping, and iOS forbids most runtime JIT (writable-executable memory is restricted for security). The result: apps launch fast, run predictably, and have small memory overhead on battery-powered devices. Android historically interpreted/JIT'd Dalvik bytecode, but modern **ART** AOT-compiles apps to native code at install time (with a JIT + profile-guided recompilation hybrid later added) precisely to get faster startup and smoother, more predictable performance and battery life. This shows AOT's sweet spot: constrained, latency- and battery-sensitive environments where warm-up and an in-process compiler are unaffordable.

5. Diagrams / Mental Models

AOT pipeline (all at BUILD time):

```
source → [ front end ] → [ optimizer ] → [ back end ] → native binary
                                                    |
                                                    ship the binary
                                                    |
RUN time:    ./app → CPU executes native code immediately (no compiler)

Performance over time: HIGH from the very first instruction (no warm-up).
```

Property	AOT	JIT
Compile time	build time	runtime
Startup	instant, full speed	fast start, ramps up
Runtime overhead	none	compiler runs in-process
Adapts to runtime data	no (PGO from training only)	yes (live profiling)
Memory	low	high
Determinism	high	lower (recompiles/pauses)
Artifact	native binary (platform-specific)	bytecode + runtime

6. Common Interview Questions

Q1. What is AOT compilation? Answer: Compiling source/bytecode fully to native machine code before execution, producing a standalone binary; no compilation happens at runtime. Common mistake: conflating it with "just compilation" without contrasting the runtime aspect vs JIT.

Q2. How does AOT differ from JIT? Answer: AOT compiles once at build time with no runtime compiler; JIT compiles at runtime using live profiles. AOT wins on startup, memory, and determinism; JIT wins on runtime adaptivity.

Q3. What are the advantages of AOT? Answer: Fast predictable startup (no warm-up), no runtime compilation overhead, lower memory, deterministic performance, and simple deployment (one binary).

Q4. What are the disadvantages of AOT? Answer: Cannot use live runtime data to specialize, no speculation/deoptimization, platform-specific binaries (need cross-compilation), and longer build times.

Q5. When would you choose AOT over JIT? Answer: Short-lived processes (CLI tools), serverless cold starts, embedded/real-time systems, mobile apps, and anywhere predictable startup and low memory matter more than peak adaptivity.

Q6. Can AOT-compiled code use runtime profiling at all? Answer: Indirectly, via Profile-Guided Optimization - run a training build, collect a profile, and recompile using it; but it cannot adapt to *live* production data like a JIT.

Q7. Is Go compiled AOT or JIT? Answer: AOT - `go build` produces a native binary with the runtime embedded; no runtime JIT.

Q8. Why do iOS apps use AOT? Answer: Fast, predictable startup and low overhead on mobile, plus security restrictions that forbid runtime JIT (writable-executable memory).

Q9. What is GraalVM native image? Answer: A tool that AOT-compiles Java bytecode into a standalone native executable, eliminating JVM warm-up and reducing startup time and memory - at the cost of some peak throughput and dynamic features.

Q10. Does AOT eliminate the need for a runtime? Answer: Not necessarily - garbage-collected AOT languages (Go, Java native image) still embed a runtime (GC, scheduler) in the binary; it just is not a *compiler*.

7. Deep-Dive Questions

D1. Why can a JIT sometimes outperform AOT despite AOT doing "more" optimization upfront?

Because the JIT sees *actual* runtime behavior - real types, hot paths, branch probabilities - and can speculate and deoptimize. AOT must produce code correct for all inputs, so it cannot assume a call site is monomorphic. PGO narrows this gap but uses a *training* profile that may not match production.

D2. What dynamic language features are hard for AOT and why? Reflection, runtime code generation (`eval`), dynamic class loading, and dynamic proxies are hard because AOT must know all reachable code at build time (closed-world assumption). GraalVM native image, for example, requires configuration to declare reflectively accessed members since it cannot discover them statically.

D3. What is the closed-world assumption in AOT and what does it enable/cost? AOT assumes the whole program is known at build time. This enables aggressive whole-program optimizations (dead-code elimination, devirtualization, smaller binaries) but forbids loading unknown code at runtime and complicates reflection/plugins.

D4. How does AOT interact with cross-compilation and portability? AOT emits native code for a specific CPU/OS, so distributing to multiple platforms requires cross-compiling separate binaries (or fat binaries). Bytecode+JIT distributes one portable artifact that adapts to whatever host runs it - a portability advantage AOT gives up for startup/predictability.

D5. Why are AOT and JIT increasingly combined? To get the best of both: AOT-precompile common code to remove warm-up (fast startup) while keeping a JIT to further optimize hot paths with live data. .NET ReadyToRun, JVM tiered AOT, and Android ART's install-time AOT plus later JIT/PGO all follow this hybrid pattern.

8. Comparison Tables

AOT vs JIT (the core interview table)

Feature	AOT	JIT
When compiled	before running (build time)	during running (runtime)
Startup	instant, full speed	fast start, warms up
Warm-up cost	none	yes
Runtime data used	training profile (PGO) only	live profiling
Speculation/deopt	no	yes
Memory footprint	low	high (in-process compiler)
Performance predictability	high	lower (recompiles, pauses)
Artifact portability	low (native)	high (bytecode)
Best for	CLIs, mobile, embedded, cold starts	long-running servers, dynamic workloads
Examples	C, Rust, Go, Swift, GraalVM native	JVM, V8, .NET CLR

AOT vs Interpreter

Feature	AOT	Interpreter
Execution	native code directly	translate per statement
Speed	high	low
Startup	instant	instant
Portability of artifact	low	high (source)

9. Common Mistakes

- Thinking AOT means "no runtime at all" (GC/runtime may still be embedded).
- Assuming AOT is always faster than JIT (JIT can win via live profiling).
- Forgetting AOT binaries are platform-specific (cross-compilation needed).
- Believing AOT cannot use profiles (PGO exists, from a training run).
- Ignoring AOT's difficulty with reflection/dynamic loading (closed-world assumption).
- Confusing AOT with one-pass compilation (AOT can be multi-pass and heavily optimizing).

10. Edge Cases / Special Cases

- **Hybrid AOT+JIT:** Android ART, .NET ReadyToRun, JVM tiered AOT precompile then JIT.
- **PGO mismatch:** if the training profile differs from production, AOT+PGO can optimize the wrong paths.
- **Reflection/eval:** need explicit configuration or are unsupported in strict AOT (GraalVM native image).

- **Binary size:** AOT whole-program compilation can bloat binaries (or shrink them via dead-code elimination), depending on linking.
- **Cross-compilation:** build ARM binaries on x86; the source is unchanged, only the back-end target differs.
- **Security:** AOT avoids writable-executable memory that JITs need, which is why locked-down platforms mandate it.

11. How to Explain in Interview

"AOT compilation does all the work at build time: it compiles the whole program to native machine code before it runs, so there is no compiler or warm-up at runtime. That gives instant, predictable startup, low memory, and deterministic performance, which is why it is used for C, Rust, Go, iOS apps, and increasingly for precompiling Java with GraalVM native image. The trade-off versus JIT is adaptivity: AOT cannot specialize on live runtime data or speculate and deoptimize, and its binaries are platform-specific. It can recover some ground with profile-guided optimization from a training run. So the rule of thumb is: short-lived, latency-sensitive, or constrained workloads favor AOT; long-running dynamic servers favor JIT - and modern systems increasingly combine both."

12. Quick Revision Notes

- AOT = compile fully to native **before running**; no runtime compiler.
- Pros: instant startup (no warm-up), no runtime overhead, low memory, deterministic, simple deploy.
- Cons: no live runtime adaptation, no speculation/deopt, platform-specific binary, slower builds, hard with reflection/eval (closed-world).
- **PGO** = AOT with a profile from a training run (partial JIT-like benefit).
- Users: C, C++, Rust, Go, Swift/iOS, GraalVM native image, .NET Native AOT, Android ART (install-time).
- **Trap:** AOT still may embed a runtime (GC); AOT is not always faster than JIT.
- Choose AOT for CLIs/mobile/embedded/cold-starts; JIT for long-running dynamic servers.

13. Practice Tasks

1. **go build** a program and measure startup time; compare with a JVM "hello world" - explain the gap.
2. Build a C program with PGO: **gcc -fprofile-generate**, run on sample input, then **-fprofile-use**; compare speed.
3. Try GraalVM **native-image** on a small Java app; measure startup and memory vs running on the JVM.
4. Cross-compile a Go binary for ARM (**GOARCH=arm64 go build**) from an x86 machine.
5. List which of these workloads favor AOT vs JIT: a 20 ms CLI tool, a 12-hour data server, an iOS game, a serverless function.
6. Explain why a JIT-style speculative inlining of a virtual call is unsafe for a pure AOT compiler.

14. Final Cheat Sheet

- **Core definition:** Compiling the whole program to native machine code before execution, producing a standalone binary with no runtime compilation.
- **Why it matters:** Instant predictable startup, low memory, determinism; ideal for CLIs, mobile, embedded, cold starts.
- **Most asked:** AOT vs JIT; when to choose AOT; PGO; why iOS uses AOT; can AOT use profiles.
- **Common comparisons:** AOT vs JIT; AOT vs interpreter; PGO vs live JIT profiling.

- **One-line answer:** "AOT compiles everything to native code before the program runs, trading a JIT's runtime adaptivity for instant startup, low memory, and predictable performance."

10. Bootstrapping a Compiler

1. Overview

Definition. Bootstrapping is the process of writing a compiler for a language *in that same language* (or using progressively more capable versions of a compiler to build itself). It answers the classic chicken-and-egg puzzle: "How do you compile the compiler for language X, written in X, if you do not yet have a compiler for X?"

Why it matters.

- It is how most serious languages (C, Go, Rust, TypeScript) end up with compilers written in themselves, which improves the compiler by dogfooding the language and lets the language's own features power the compiler.
- Understanding it clarifies the difference between the language a compiler *compiles*, the language it is *written in*, and the machine it *runs on* - the three axes captured by **T-diagrams**.

Where it is used in real systems. GCC (C compiler written in C), the Go compiler (originally C, now written in Go), **rustc** (Rust compiler written in Rust), the TypeScript compiler (written in TypeScript), and historically the first self-compiling compilers (a Lisp compiler in Lisp, an Algol compiler, etc.).

Why interviewers ask about it. It is a beautiful test of clear thinking about the chicken-and-egg problem, the three languages involved in any compiler (source, implementation, target), and the staged process by which a compiler comes to compile itself.

2. Core Idea

Break the chicken-and-egg cycle with a smaller starting point. You cannot compile an X-compiler-written-in-X without an X compiler, so you *start* with something you already have: another language, a subset, or an existing compiler. Once you have *any* working compiler for X, you use it to compile the "real" compiler (written in X), and from then on the compiler builds itself.

Intuition. To build the first metal lathe, you cannot use a metal lathe - you shape the first one crudely by hand, then use it to make a better lathe, then use *that* to make an even better one. Each generation is built by the previous. A compiler bootstraps the same way.

Real-world analogy - learning to make tools. Early humans used stone tools to make better stone tools, then metal tools, then machines that make machines. Nobody needed a factory to build the first hammer; you start crude and improve. Bootstrapping is "the compiler pulls itself up by its own bootstraps".

The three languages in every compiler (crucial mental model). A compiler is described by three things:

- **S = Source language** it compiles (what it reads).
- **I = Implementation language** it is written in (what its own code is).
- **T = Target language** it produces (what it writes).

A **T-diagram** notates a compiler as "compiles S, written in I, targets T".

Classic bootstrap sequence (for a self-hosting compiler of language X targeting machine M):

1. Write a **simple compiler for a subset of X** in an existing language (say C). Call it **Compiler0**. It compiles (subset of X) → M, and it runs on M because C was already compilable on M.
2. Write the **full X compiler in X itself** (source code, not yet runnable). Call it **Compiler1-source**.
3. Compile **Compiler1-source** using **Compiler0** → you get **Compiler1**, a runnable X compiler on M, but built by the limited **Compiler0**.
4. Now recompile **Compiler1-source** using **Compiler1** itself → **Compiler2**. Since **Compiler1** is a full, optimizing X compiler, **Compiler2** is better/faster.
5. Compiling **Compiler2-source** with **Compiler2** should reproduce an identical binary - the **fixed-point / triple test** confirming the bootstrap is stable.

3. Important Subtopics

3.1 The chicken-and-egg problem

- **What:** You need an X compiler to compile an X compiler written in X.
- **Why it matters:** It is the whole motivation; the solution is to start from a different, already-available tool.
- **Interview angle:** "How do you compile the first compiler for a language written in itself?" - start with another language or a subset.

3.2 The three languages (Source / Implementation / Target) and T-diagrams

- **What:** Distinguish what the compiler compiles, what it is written in, and what it emits.
- **Why it matters:** Bootstrapping is confusing until you separate these three axes; T-diagrams make composition explicit.
- **Interview angle:** Be able to label S, I, T for **rustc**, **gcc**, **tsc**.

3.3 Starting points for the first compiler

- **What:** Options - write it in an existing language, write it for a *subset* of the language, or hand-write/assemble a minimal compiler.
- **Why it matters:** Every bootstrap needs a seed; the seed need not be pretty, just correct enough.
- **Example:** Go's first compiler was written in C; later Go was used to rewrite it in Go.
- **Interview angle:** Subset bootstrapping vs foreign-language bootstrapping.

3.4 Staged self-compilation and the fixed-point (triple) test

- **What:** Compile the self-hosted compiler with the previous stage, then with itself, and check that compiling again reproduces an identical binary.
- **Why it matters:** Proves the bootstrap is correct and stable; catches nondeterminism or bugs.
- **Interview angle:** Why compile it *twice*? To verify self-consistency (stage2 == stage3).

3.5 Cross-compilation as a bootstrap tool

- **What:** Use a compiler on an existing machine to produce a compiler binary for a *new* machine/architecture.

- **Why it matters:** Bootstraps a compiler onto hardware that has no compiler yet.
- **Interview angle:** Porting a language to a new CPU often uses cross-compilation first, then self-hosting on the target.

4. Real-World Example

The Go compiler's bootstrap history. Go's compiler was originally written in **C**. That C-based compiler compiled Go programs, including the Go source of a *new* Go compiler written in Go. Once that Go-in-Go compiler could be built (by the C compiler) and could correctly compile itself, the team **dropped the C implementation entirely** - modern Go is compiled by a Go compiler written in Go. To build Go today, you need an existing Go compiler (a "bootstrap toolchain"); the build compiles the current source with the previous version, then with itself, verifying the result is identical. This is exactly the staged bootstrap: seed with another language (C), migrate the implementation to the target language (Go), then let it compile itself. Rust did the same starting from an OCaml-written compiler before **rustc** became self-hosting.

5. Diagrams / Mental Models

The three languages of a compiler (T-notation):

compiles S

targets T

written in I

Bootstrap staging (X = new language, C = seed language, M = machine):

Stage 0: [subset-of-X compiler, written in C, targets M] --built by existing C toolchain

| use it to compile...

▼

Stage 1: X-compiler source (written in X) —Stage0→ Compiler1 (runs on M)

| recompile the SAME source with Compiler1...

▼

Stage 2: X-compiler source (written in X) —Compiler1→ Compiler2 (better/faster)

| recompile again with Compiler2...

▼

Stage 3: should produce a binary IDENTICAL to Compiler2 → ✔ fixed-point test passes

Symbol	Meaning	Example (rustc)
S (source)	language it compiles	Rust
I (implementation)	language it is written in	Rust
T (target)	language it emits	machine code / LLVM IR

6. Common Interview Questions

Q1. What is bootstrapping a compiler? Answer: Writing a compiler for a language in that same language, and using a staged process (starting from an existing compiler/subset) to make it compile itself. Common

mistake: describing it as "starting the OS" or confusing it with OS boot.

Q2. What is the chicken-and-egg problem in bootstrapping? Answer: You need an X compiler to compile an X compiler written in X; you break the cycle by starting with a different language or a language subset.

Q3. What are the three languages associated with a compiler? Answer: Source (what it compiles), implementation (what it is written in), and target (what it emits).

Q4. How do you build the very first compiler for a new self-hosted language? Answer: Write an initial compiler in an existing language (or for a subset of the new language), use it to compile the real compiler written in the new language, then let that compiler compile itself.

Q5. Why compile the self-hosting compiler more than once (staging)? Answer: To improve it (a full compiler recompiles its own source better than the seed did) and to verify stability via the fixed-point test - stage 2 and stage 3 binaries should be identical.

Q6. What is the fixed-point / triple test? Answer: Compiling the compiler's source with the newly built compiler should reproduce a byte-identical binary; if stage2 == stage3, the bootstrap is self-consistent.

Q7. Give a real example of a bootstrapped compiler. Answer: GCC (C in C), Go (seeded in C, now Go in Go), Rust (seeded in OCaml, now Rust in Rust), TypeScript (TS in TS).

Q8. How is cross-compilation used in bootstrapping? Answer: To create a compiler binary for a new machine using a compiler on an existing machine, so the new hardware can then host a self-compiling compiler.

Q9. What are the benefits of a self-hosted (bootstrapped) compiler? Answer: The language dogfoods itself (bugs and ergonomics surface), the compiler can use the language's own features, and it proves the language is expressive enough to build serious software.

Q10. Is bootstrapping the same as self-hosting? Answer: Closely related: self-hosting is the *state* of a compiler being written in the language it compiles; bootstrapping is the *process* of getting there (and of building it from a seed each time).

7. Deep-Dive Questions

D1. Walk through the exact stages of a self-hosting bootstrap and what each verifies. Stage 1: build the new compiler's source with an older/seed compiler - proves it *compiles*. Stage 2: rebuild the same source with the stage-1 binary - proves it can compile *itself* and applies its own optimizations. Stage 3: rebuild again with the stage-2 binary and compare to stage 2 - if identical, it proves determinism and that the compiler is a *fixed point* (no lingering influence from the seed).

D2. What is Ken Thompson's "Trusting Trust" attack and how does it relate to bootstrapping?

Thompson showed a compiler binary could be modified to insert a backdoor whenever it compiles a login program, *and* to reinsert that malicious logic whenever it compiles a compiler - so the malice survives even after the source is clean and the compiler is rebuilt from itself. It highlights that a bootstrapped compiler's trust ultimately rests on the trust of the *binary seed*, not just the source, motivating reproducible builds and "diverse double-compiling".

D3. What is Diverse Double-Compiling (DDC) and what problem does it solve? DDC (David A. Wheeler) defends against Trusting Trust: compile the compiler's source with a *different, independent* trusted compiler to

get compiler A, then use A to recompile the real compiler; if the result matches the official binary, no hidden backdoor was injected by the original seed. It verifies the binary corresponds to its source.

D4. Why not just always write the compiler in an existing language and skip self-hosting? You could, but self-hosting yields dogfooding (the team feels the language's rough edges), lets the compiler exploit the language's own abstractions, removes the dependency on a foreign toolchain, and demonstrates the language is mature. The seed language is a means to that end, not the destination.

D5. How do you bootstrap a language onto a brand-new CPU architecture with no existing tools?

Cross-compile: run the existing (self-hosting) compiler on a known machine but configure its back end to target the new CPU, producing a compiler binary that runs on the new machine. Copy it over; from then on the new machine can self-host. Historically the very first compilers on new hardware were partly hand-assembled.

8. Comparison Tables

The three languages of a compiler

Axis	Question it answers	rustc	tsc	gcc
Source (S)	what does it compile?	Rust	TypeScript	C
Implementation (I)	what is it written in?	Rust	TypeScript	C
Target (T)	what does it emit?	machine code	JavaScript	machine code

Bootstrapping vs Self-hosting

	Bootstrapping	Self-hosting
Nature	a process	a state/property
Meaning	building a compiler from a seed (often to make it self-compile)	the compiler is written in the language it compiles
Question	"how did we get here?"	"is it written in itself?"

Seed strategies

Strategy	Seed	Example
Foreign language	another language's compiler	Go (in C), Rust (in OCaml)
Language subset	a simpler subset compiler	many early languages
Cross-compilation	compiler on another machine	porting to new CPU

9. Common Mistakes

- Confusing compiler bootstrapping with OS booting (unrelated concepts).
- Not separating the three languages (source vs implementation vs target).
- Thinking self-hosting is impossible because of the chicken-and-egg problem (a seed breaks it).
- Believing one compile is enough (staging/fixed-point verification needs multiple compiles).

- Assuming clean source guarantees a clean compiler (Trusting Trust: the binary seed matters).
- Using "bootstrapping" and "self-hosting" as exact synonyms (process vs state).

10. Edge Cases / Special Cases

- **Trusting Trust:** a malicious seed binary can perpetuate a backdoor across rebuilds despite clean source.
- **Reproducible builds:** required so the fixed-point test (`stage2 == stage3`) is byte-for-byte verifiable.
- **Nondeterminism** (timestamps, hash-map ordering, addresses) can break the fixed-point test even for a correct compiler.
- **Bootstrap toolchain requirement:** building Go/Rust needs a *previous* version of the same compiler.
- **Porting to new hardware:** first compiler often cross-compiled or partly hand-assembled.
- **Multi-language seeds:** some compilers pass through several implementation languages historically before self-hosting.

11. How to Explain in Interview

"Bootstrapping solves a chicken-and-egg problem: how do you compile a compiler for language X that is itself written in X, when you have no X compiler yet? You break the cycle with a seed - write an initial compiler in an existing language or for a subset of X. Then you use that seed to compile the real compiler written in X, and from then on the compiler can compile itself. Usually it is staged: build it with the seed, rebuild it with itself, and rebuild once more to check the binary is identical - the fixed-point test that proves the bootstrap is stable. The key clarity is separating the three languages a compiler involves: the source it compiles, the language it is written in, and the target it emits. Go did exactly this - seeded in C, then rewritten in Go, and now Go compiles Go."

12. Quick Revision Notes

- Bootstrapping = building a compiler from a **seed** so it can eventually compile itself.
- Chicken-and-egg: need X compiler to compile X-in-X → break with another language or a subset.
- **Three languages:** Source (compiles), Implementation (written in), Target (emits).
- Staging: seed → build stage1 → rebuild stage2 (with itself) → stage3 identical = **fixed-point test**.
- Examples: GCC (C/C), Go (C→Go), Rust (OCaml→Rust), TypeScript (TS/TS).
- Cross-compilation bootstraps onto new hardware.
- **Trap:** clean source ≠ clean compiler (Trusting Trust); bootstrapping (process) ≠ self-hosting (state).

13. Practice Tasks

1. Label the source, implementation, and target languages for gcc, rustc, tsc, and javac.
2. Draw a T-diagram for a compiler that compiles X, is written in C, and targets x86.
3. Write out the staged bootstrap for a new language "Zeta" seeded in Python, listing each stage's input compiler and output.
4. Explain why stage2 and stage3 binaries must be identical and what a mismatch would indicate.
5. Research and summarize in 3 lines how Go migrated from a C compiler to a self-hosted Go compiler.
6. Explain in your own words the Trusting Trust attack and how Diverse Double-Compiling defends against it.

14. Final Cheat Sheet

- **Core definition:** Building a compiler from a seed (another language or a subset) so a language's compiler can be written in, and compile, itself.
 - **Why it matters:** Solves the chicken-and-egg problem; enables dogfooded, self-hosted compilers.
 - **Most asked:** Chicken-and-egg problem; the three languages; staging and the fixed-point test; a real example (Go/Rust).
 - **Common comparisons:** Bootstrapping (process) vs self-hosting (state); source vs implementation vs target.
 - **One-line answer:** "Bootstrapping breaks the chicken-and-egg problem by using a seed compiler to build a language's compiler until that compiler can compile itself, verified by a staged fixed-point test."
-

11. Self-Hosting Compilers

1. Overview

Definition. A **self-hosting compiler** is a compiler that is *written in the very language it compiles*. For example, a C compiler written in C, a Rust compiler (**rustc**) written in Rust, or the TypeScript compiler written in TypeScript. It compiles its own source code.

Why it matters.

- It is widely seen as a milestone of language maturity: the language is expressive and stable enough to build one of the most demanding programs (a compiler) in itself.
- It creates a virtuous cycle - the compiler *dogfoods* the language, so the team feels every rough edge, and improvements to the language immediately benefit the compiler.
- It removes the dependency on a foreign toolchain once achieved.

Where it is used in real systems. GCC (C compiler in C), Clang (C++ compiler in C++), **rustc** (Rust in Rust), the Go compiler (Go in Go), the TypeScript compiler (TS in TS), many Lisp/Scheme compilers, and historically the first self-hosting compiler is often credited to early Lisp/Algol/NELIAC efforts.

Why interviewers ask about it. It pairs naturally with bootstrapping and tests whether you understand the difference between a compiler's *implementation language*, *source language*, and *target*, plus why self-hosting is desirable and how it is achieved and verified.

2. Core Idea

The compiler eats its own dog food: it compiles the source code it is itself written in. Once a compiler for language X is written in X, you can use the current compiler binary to build the next version from source, indefinitely - each version compiles the next.

Intuition. A language becomes "self-sufficient" when it can build its own toolchain. Self-hosting means you no longer need any other language to maintain the compiler; the language sustains itself.

Real-world analogy - a factory that builds its own machines. A truly self-sufficient factory can manufacture the very machines it runs on. If a machine wears out, the factory builds a replacement using its existing machines. A self-hosting compiler is a toolchain that can reproduce (and improve) itself using itself.

Small example (conceptual).

```
rustc (a binary) --compiles--> rustc's own Rust source --> new rustc binary  
                                (written in Rust)
```

The implementation language (Rust) equals the source language (Rust). Contrast a *non*-self-hosting compiler: early **rustc** was written in **OCaml**, so OCaml (implementation) != Rust (source) - not self-hosting until it was rewritten in Rust.

Relationship to bootstrapping.

- **Self-hosting** = the *property/state*: "the compiler is written in its own language".
- **Bootstrapping** = the *process* used to *reach* self-hosting (seed compiler → compile the self-hosted source → let it compile itself) and to *rebuild* it thereafter. So every self-hosting compiler was bootstrapped, but bootstrapping is the journey and self-hosting is the destination.

3. Important Subtopics

3.1 Implementation language = source language

- **What:** The defining property - the compiler's own code is written in the language it compiles.
- **Why it matters:** This is the exact criterion for "self-hosting".
- **Example:** **gcc** in C, **tsc** in TypeScript.
- **Interview angle:** "What makes a compiler self-hosting?" - I equals S.

3.2 Dogfooding and language maturity

- **What:** Building the compiler in its own language stress-tests the language on a large, real program.
- **Why it matters:** Surfaces missing features, bugs, and ergonomic problems early; a badge of maturity.
- **Interview angle:** Why is self-hosting considered a milestone? - proves expressiveness and stability.

3.3 The bootstrap toolchain / reproducing the compiler

- **What:** To build a self-hosting compiler from source, you need a *previous* working binary of it (or a seed); builds are staged.
- **Why it matters:** You cannot compile the source without an existing compiler; hence a bootstrap binary is required.
- **Example:** Building Rust needs a prior **rustc**; building Go needs a prior Go toolchain.
- **Interview angle:** "How do you build a self-hosting compiler from scratch?" - via bootstrapping from a seed.

3.4 Verification: staged builds and the fixed-point test

- **What:** Compile the source with the old compiler (stage1), then with itself (stage2), then again (stage3); stage2 and stage3 should be identical.
- **Why it matters:** Confirms the self-hosted compiler is correct, deterministic, and independent of the seed.
- **Interview angle:** Why identical binaries? - proves a stable fixed point.

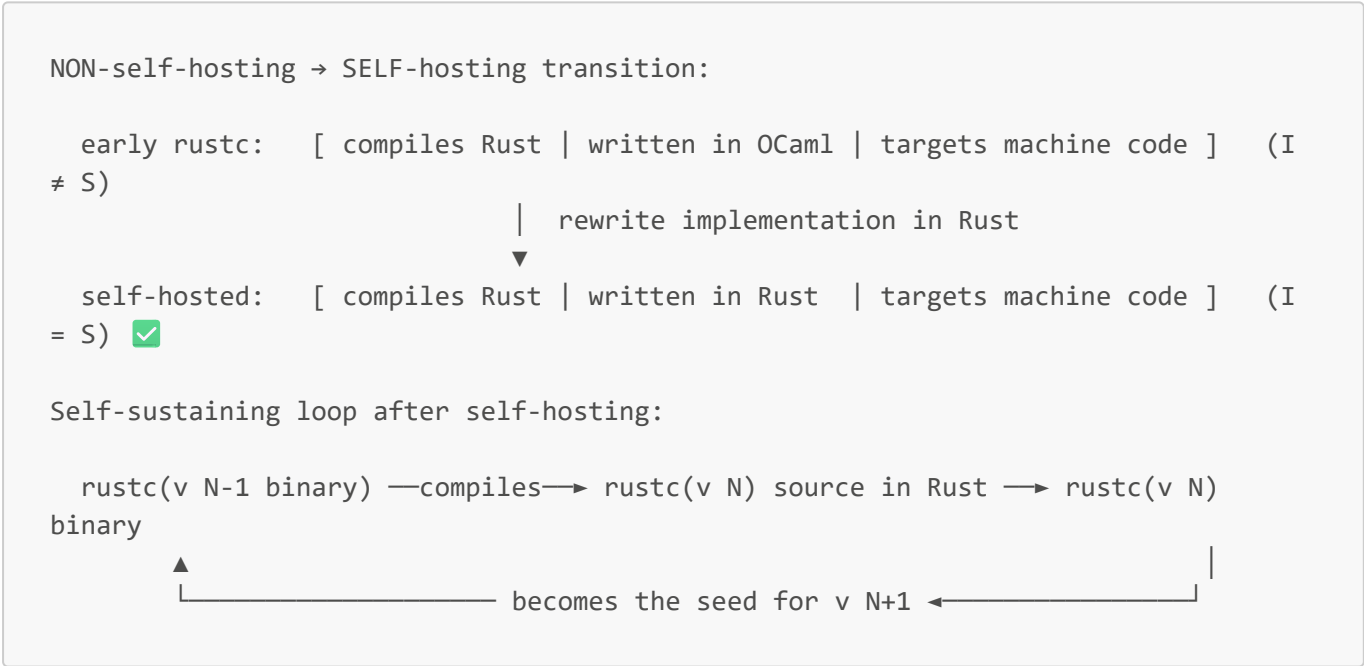
3.5 Trust and reproducibility concerns

- **What:** Since the compiler builds itself from a binary seed, a compromised seed could persist (Trusting Trust); reproducible builds and diverse double-compiling mitigate this.
- **Why it matters:** Security and supply-chain integrity of self-hosting toolchains.
- **Interview angle:** "What is the security risk of self-hosting?" - trust anchored in the binary, not just source.

4. Real-World Example

rustc becoming self-hosting. The first Rust compiler was written in **OCaml** because no Rust compiler existed to write it in Rust. As the language matured, the team rewrote the compiler in **Rust itself**. From that point **rustc** was self-hosting: each release of Rust is compiled by the *previous* release of **rustc** (the "bootstrap compiler"), and the freshly built compiler then recompiles the source to verify consistency. This is why installing Rust from source requires a stage0 **rustc** binary to begin. The payoff is direct dogfooding - the Rust compiler is one of the largest Rust programs in existence, so anything painful about writing Rust is felt immediately by the people who can fix it, and every new language feature can be used to improve the compiler. Go, GCC, Clang, and TypeScript all live in this same self-sustaining loop.

5. Diagrams / Mental Models



Property	Non-self-hosting	Self-hosting
Implementation language	different from source	same as source
Needs foreign toolchain	yes	no (after bootstrap)
Dogfoods the language	no	yes
Example	early rustc (OCaml)	current rustc (Rust)

6. Common Interview Questions

Q1. What is a self-hosting compiler? Answer: A compiler written in the same language it compiles, so it can compile its own source code (e.g. a C compiler written in C). Common mistake: confusing "self-hosting" with

"self-executing" or with bootstrapping the OS.

Q2. What is the difference between self-hosting and bootstrapping? Answer: Self-hosting is the property that the compiler is written in its own language; bootstrapping is the process (starting from a seed) used to build/reach that self-hosting state.

Q3. Why is self-hosting considered a sign of language maturity? Answer: A compiler is a large, demanding program; being able to write and maintain it in the language proves the language is expressive, stable, and practical.

Q4. How do you build a self-hosting compiler for the first time? Answer: Bootstrap it - write the initial compiler in another language (or a subset), use it to compile the self-hosted source, then let that compiler compile itself.

Q5. Why do you need an existing binary to build a self-hosting compiler from source? Answer: Because the source is written in the target language, you need a working compiler for that language (a bootstrap/stage0 binary) to compile it.

Q6. Give three examples of self-hosting compilers. Answer: GCC (C in C), rustc (Rust in Rust), the Go compiler (Go in Go); also Clang (C++), tsc (TypeScript).

Q7. What is the benefit of the compiler dogfooding its own language? Answer: The maintainers experience the language's weaknesses firsthand on a real large codebase, driving fixes and improvements, and the compiler can use the language's own features.

Q8. How is a self-hosting compiler build verified? Answer: Via staged builds - compile with the old compiler, then with itself, and check that recompiling reproduces an identical binary (fixed-point test).

Q9. What security concern is unique to self-hosting? Answer: Ken Thompson's Trusting Trust - a malicious compiler binary can perpetuate a backdoor through self-rebuilds even with clean source; reproducible builds and diverse double-compiling mitigate it.

Q10. Can a compiler be self-hosting but not able to bootstrap from scratch without a binary? Answer: Yes - self-hosting compilers generally require a prior binary seed to build from source; that is normal and expected.

7. Deep-Dive Questions

D1. Why exactly is a prior binary unavoidable for a self-hosting compiler, and how is the very first one made? Because the source cannot compile itself without an existing compiler for its language. The very first compiler is made by *not* being self-hosting initially - it is written in another language or a subset (the seed), used once to produce the first self-hosted binary, after which the language can sustain itself. This is the bootstrapping step.

D2. What can go wrong in a self-hosting build and how is it detected? Bugs in the compiler can silently miscompile its own source, producing a subtly broken next-generation compiler. Staged builds detect many such issues: if stage2 (built by stage1) and stage3 (built by stage2) differ, something is nondeterministic or wrong. Reproducible-build discipline (no timestamps/addresses in output) is required for this check to be meaningful.

D3. How does self-hosting interact with adding a new language feature the compiler itself wants to use? There is a chicken-and-egg at the feature level: you must first ship a compiler that *supports* the new feature (built without using it), then a later compiler version can *use* the feature in its own source, compiled by the previous version. This is why compilers stage feature adoption a release behind their introduction.

D4. Is self-hosting always desirable? When might you avoid it? Not always. Very small/embedded DSLs, or languages intentionally kept minimal, may never justify a self-hosted compiler. Also, during early development, writing the compiler in a mature language (with good tooling/libraries) is faster and less risky than self-hosting prematurely. Self-hosting is worth it once the language is stable and the dogfooding/maintenance benefits outweigh the bootstrap complexity.

D5. How do reproducible builds and Diverse Double-Compiling secure a self-hosting toolchain?

Reproducible builds guarantee the same source yields byte-identical binaries, so anyone can independently verify the official binary matches the source. Diverse Double-Compiling recompiles the compiler's source with an *independent* trusted compiler and checks the result matches the official binary, defeating a Trusting-Trust backdoor hidden only in the original binary seed.

8. Comparison Tables

Self-hosting vs Non-self-hosting

Feature	Self-hosting	Non-self-hosting
Implementation language	same as source ($I = S$)	different ($I \neq S$)
Compiles its own source	yes	no
Foreign toolchain dependency	none (after bootstrap)	yes
Dogfoods the language	yes	no
Example	rustc (Rust), gcc (C)	early rustc (OCaml), CPython (C)

Self-hosting vs Bootstrapping

	Self-hosting	Bootstrapping
Type	property/state	process
Question	"written in its own language?"	"how was it built from a seed?"
Relationship	the goal	the path to the goal (and to rebuild it)

Language / implementation examples

Compiler	Compiles	Written in	Self-hosting?
gcc	C	C	Yes
clang	C/C++	C++	Yes
rustc (now)	Rust	Rust	Yes
rustc (early)	Rust	OCaml	No

Compiler	Compiles	Written in	Self-hosting?
CPython	Python	C	No
tsc	TypeScript	TypeScript	Yes

9. Common Mistakes

- Confusing self-hosting (compiler written in its own language) with bootstrapping (the process to build it).
- Thinking self-hosting means it needs *no* binary to build from source (it needs a prior/seed binary).
- Believing CPython is self-hosting (it is written in C, not Python - so it is not).
- Assuming self-hosting guarantees correctness or security (Trusting Trust shows otherwise).
- Treating self-hosting as mandatory for every language (it is a milestone, not a requirement).
- Mixing up implementation language with target language.

10. Edge Cases / Special Cases

- **Partially self-hosting:** a compiler may have parts in another language (runtime in C, front end in the language itself).
- **Feature staging:** the compiler adopts a new language feature one release *after* shipping support for it.
- **Trusting Trust:** trust ultimately rests on the binary seed, not only the source.
- **Reproducible builds** are needed for the fixed-point verification to be meaningful.
- **Interpreters vs compilers:** CPython is a self-*interpreting* language's reference implementation written in C - not self-hosting; PyPy (Python interpreter written in RPython, a Python subset) is closer to self-hosting.
- **Cross-language runtimes:** even self-hosted compilers may depend on a C/assembly runtime and a linker they do not implement.

11. How to Explain in Interview

"A self-hosting compiler is one written in the same language it compiles - like GCC being written in C or rustc being written in Rust - so it can compile its own source. It is considered a milestone of language maturity because a compiler is a big, demanding program, and building it in the language dogfoods every feature. It is closely tied to bootstrapping: bootstrapping is the process that gets you there - you seed with another language, compile the self-hosted source once, and then the compiler can compile itself, verified by staged builds where recompiling reproduces an identical binary. One subtlety: even a self-hosting compiler needs a prior binary to build from source, and there is a famous security caveat, Thompson's Trusting Trust, that trust rests on the seed binary, not just the clean source."

12. Quick Revision Notes

- Self-hosting = compiler **written in the language it compiles** (Implementation = Source).
- It is a **milestone of maturity** (dogfooding, expressiveness, no foreign toolchain).
- **Self-hosting = state; bootstrapping = process** to reach/rebuild it.
- Needs a prior/seed **binary** to build from source; verified by staged **fixed-point test**.
- Examples: gcc (C), clang (C++), rustc (Rust), Go (Go), tsc (TS). Not self-hosting: CPython (C).
- Early rustc was in OCaml (non-self-hosting) → rewritten in Rust (self-hosting).
- **Trap:** self-hosting $\neq$ needs no binary; clean source $\neq$ trustworthy binary (Trusting Trust).

13. Practice Tasks

1. For gcc, clang, rustc, CPython, and tsc, state whether each is self-hosting and justify by naming its implementation language.
2. Explain why CPython is not self-hosting and what it would take for it to be.
3. Outline the steps to make a hypothetical language "Nova" self-hosting, starting from a Python-written seed.
4. Describe the staged build of a self-hosting compiler and why stage2 must equal stage3.
5. Explain, with the rustc/OCaml example, the difference between self-hosting and bootstrapping.
6. In two lines, describe how reproducible builds help verify a self-hosting compiler is not backdoored.

14. Final Cheat Sheet

- **Core definition:** A compiler written in the same language it compiles, so it can compile its own source (implementation language = source language).
- **Why it matters:** Milestone of language maturity; dogfooding; toolchain independence.
- **Most asked:** What makes a compiler self-hosting; self-hosting vs bootstrapping; why a prior binary is needed; examples; Trusting Trust.
- **Common comparisons:** Self-hosting vs non-self-hosting; self-hosting (state) vs bootstrapping (process).
- **One-line answer:** "A self-hosting compiler is written in the language it compiles, so it compiles its own source - reached via bootstrapping and prized as a sign of language maturity."

Cross-Topic Summary Table

Topic	One-line takeaway
Compiler vs Interpreter	Translate all before running vs translate and run per statement; most languages are hybrids.
Compiler Phases	Lexical → Syntax → Semantic → Intermediate → Optimization → Code Gen, with symbol table + error handler across all.
Front/Middle/Back End	Language-specific front, neutral optimizer middle, machine-specific back, joined by an IR (N+M not N*M).
Source & Target Language	Compiler = function Source → Target preserving semantic equivalence; target need not be machine code.
Symbol Table	The compiler's identifier database (type, scope, offset) used by every phase.
Error Handling	Detect, report, recover (panic mode) per phase so many errors surface per run.
One-pass vs Multi-pass	Single fast traversal vs multiple traversals enabling global optimization and forward references.
JIT	Compile hot code to native at runtime using profiling and speculation; fast start + fast steady state.
AOT	Compile fully to native before running; instant startup, low memory, predictable, less adaptive.

Topic	One-line takeaway
Bootstrapping	Use a seed compiler to reach a self-compiling compiler; three languages = source, implementation, target.
Self-hosting	A compiler written in the language it compiles; a milestone of maturity reached via bootstrapping.